

Benchmarking the Recommendation Spectrum: An Empirical Analysis of Architectural Inductive Biases Across Randomized and Temporal Evaluation Regimes[★]

Le Van Tuan Anh^{a,c}, Nguyen Duy Tung^{a,c}, Le Tuan Hung^{b,c}, Tran Hong Ha^c, Phung Thanh Quang^c

^a*DreamAI Lab*

^b*ML Lab*

^c*Hanoi University of Science and Technology*

Abstract

Modern recommender systems have transitioned rapidly from classical heuristic-driven algorithms to deep, graph-based, and sequential neural architectures. However, the performance of these models is deeply intertwined with their architectural inductive biases and the specific validation protocols used to evaluate them. In this work, we present a systematic, head-to-head empirical evaluation tracking the paradigm shift of recommendation methodologies from foundational models to state-of-the-art architectures. We evaluate eight distinct approaches spanning the entire evolutionary spectrum: Content-Based, Memory-Based Collaborative Filtering, Hybrid methods, Matrix Factorization, Neural Collaborative Filtering (NCF), Two-Tower architectures, LightGCN, and SASRec. To guarantee a robust architectural stress test, these models are rigorously benchmarked on the MovieLens-1M dataset across two distinct validation regimes representing fundamentally different operational scenarios: a standard 80/20 random split strategy and a strict, sequence-aware temporal leave-one-out (1-vs-all) evaluation protocol. Our findings expose critical trade-offs between architectural complexity and ranking robustness, demonstrating how model rankings dramatically shift under the influence of temporal data constraints versus randomized item allocation. We provide concrete insights into when lightweight, classical paradigms remain highly competitive against parameter-heavy neural counterparts.

1. Introduction

Recommender systems have become a central component of modern information access, helping users navigate large collections of movies, products, music, news, and online content. Instead of relying only on explicit search queries, recommendation models infer user preferences from historical interactions, item attributes, and behavioral patterns, then rank candidate items according to their estimated relevance. This makes recommendation both a practical engineering problem and a challenging machine learning problem, because models must capture sparse feedback, changing user interests, popularity bias, and the trade-off between accuracy and scalability [1].

The field has evolved through several major modeling paradigms. Early collaborative filtering methods exploit similarities among users or items based on historical ratings and interactions, making them simple and interpretable but often sensitive to sparsity and cold-start conditions [2]. Matrix factorization methods then introduced a compact latent-factor view of recommendation, representing users and items in a shared embedding space and becoming a strong classical baseline for large-scale recommender systems [5]. More recently, neural recommender systems have

[★]Source Code Repository: <https://github.com/tuananhleovan/ml-rec-sys>

extended this idea by replacing fixed similarity functions with learnable nonlinear architectures, allowing models such as Neural Collaborative Filtering to capture more flexible user-item interaction patterns [15].

In parallel, the growth of implicit-feedback data has motivated ranking-oriented and retrieval-oriented approaches. In many real systems, user preferences are not expressed as explicit ratings but as clicks, views, purchases, or watch histories. This requires models to learn from positive-only or weakly labeled feedback and to evaluate recommendation quality using top- K ranking metrics rather than only rating prediction error [6]. Large-scale industrial recommenders commonly adopt a two-stage pipeline, where an efficient retrieval model first generates candidate items and a more expensive ranking model then refines the final list [13]. Two-Tower architectures follow this retrieval design by learning separate user and item encoders whose embeddings can be compared efficiently through dot product or cosine similarity.

Beyond classical and neural collaborative filtering, recent recommender systems also incorporate graph and sequential inductive biases. Graph-based models such as LightGCN simplify graph convolution for collaborative filtering and directly propagate user-item signals over the interaction graph [7]. Sequential models such as SASRec use self-attention to model ordered user histories, making them suitable for settings where the next recommendation depends strongly on recent behavior [16]. These architectural differences suggest that there is no universally best recommender model; performance depends on the dataset, feedback type, evaluation protocol, and deployment objective.

This project provides a compact empirical synthesis of representative recommender-system families. Our group compares methods spanning content-based recommendation, memory-based collaborative filtering, hybrid approaches, matrix factorization, Neural Collaborative Filtering, Two-Tower retrieval, LightGCN, and SASRec on the MovieLens-1M benchmark [8]. We evaluate the models under both random and temporal splitting regimes in order to study how validation design changes the observed ranking quality. Through this comparison, the project aims not only to report performance numbers, but also to clarify the practical strengths and limitations of different recommender architectures.

2. Dataset

All models in this study were evaluated on the widely adopted **MovieLens-1M** (ML-1M) dataset [8], a benchmark corpus for evaluating collaborative filtering algorithms. The dataset comprises approximately 1 million explicit ratings (ranging from 1 to 5 stars) applied to roughly 4,000 movies by 6,000 distinct users. Alongside interaction data, ML-1M also provides valuable side information, including user demographics (age, gender, occupation, and zip code) and movie metadata (title, release year, and genres), which are particularly crucial for evaluating our Content-Based and Hybrid models.

To align with the prevalent implicit feedback scenario targeted by most modern recommender systems (e.g., clicks, views, or purchases), we transformed the explicit ratings into binary interaction signals. Specifically, we adopted a thresholding approach, retaining only ratings of 3.0 or higher as positive implicit feedback. Ratings below this threshold, along with unobserved interactions, were treated as negative signals. This conversion simulates a realistic environment where we model a user’s affinity or preference towards an item rather than predicting exact rating scores.

Furthermore, each interaction in the ML-1M dataset is accompanied by a timestamp. This temporal dimension is critical for our evaluation methodology, as it enables the implementation of a rigorous temporal leave-one-out (1-vs-all) splitting strategy, allowing us to evaluate the models’ performance in predicting a user’s future interactions based on their chronological historical sequence.

A defining characteristic of the ML-1M dataset is its relatively high interaction density compared to large-scale industrial recommender systems. With an overall matrix density of roughly 4.19%, the user-item bipartite interaction graph is highly connected, minimizing the extreme cold-start challenges often encountered in real-world platforms. This rich structural information provides an excellent and stable testbed for exploring deep collaborative filtering techniques and isolating the architectural inductive biases of the models without being overwhelmed by extreme data

sparsity.

3. Modelling and Algorithm

3.1. Content-Based Filtering

3.1.1. Overview

Content-Based Filtering (CBF) is a foundational paradigm in recommender systems that suggests items based on the features of the items a user has historically interacted with and liked. Unlike Collaborative Filtering (CF) methodologies, which rely on the collective interaction patterns of a user community, CBF operates on an assumption of user independence. It constructs a highly personalized profile for each individual by learning their specific affinities toward explicit item attributes, such as genres, directors, or keywords [3].

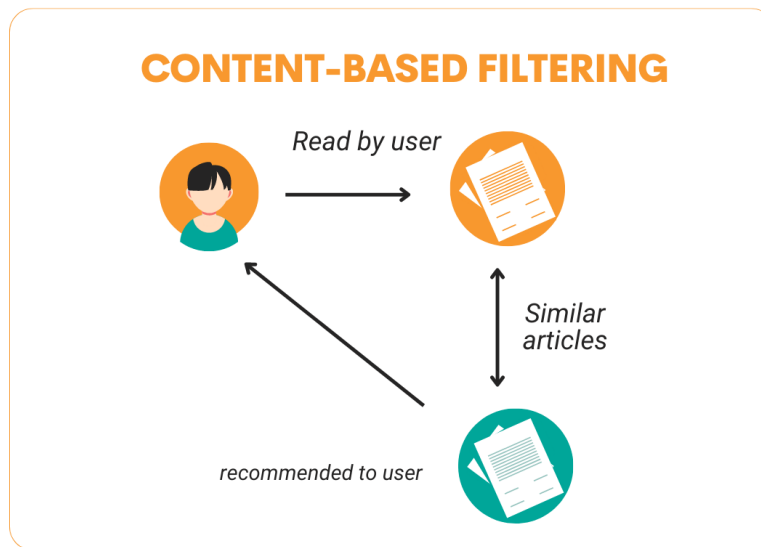


Figure 1: Core idea of CBF.

As illustrated by the independent nature of the approach, the architecture bypasses shared global parameters in favor of decoupled, user-specific models. In the context of the MovieLens-1M dataset:

- **Item Representation:** Movies are stripped of their collaborative context and represented purely by their explicit metadata—specifically, their genre classifications transformed into continuous TF-IDF vectors.
- **User Modeling:** A distinct, independent linear regression model is instantiated for every user, learning a personalized weight vector that captures their specific genre preferences based solely on their own historical ratings.

The defining strategic advantage of this architecture lies in its extreme interpretability and complete immunity to the cold-start problem for new items. Because recommendations are driven entirely by explicitly engineered features rather than topological interaction graphs, the model can instantly recommend a newly released movie as long as its metadata is available. Within this framework, the content-based approach demonstrates how classical machine learning techniques can efficiently capture direct user-item relevance without relying on complex, over-parameterized neural networks.

3.1.2. Pipeline

The proposed system follows a structured retrieval and training pipeline designed for highly parallelizable content-based filtering:

- **Data Preprocessing:** Raw MovieLens data is parsed to extract movie genres. The discrete genre strings are binarized and transformed into continuous, normalized TF-IDF feature vectors. Users and items are mapped to zero-indexed latent spaces.
- **Model Training:** For each user, the system isolates their interaction history and trains an independent Ridge Regression model. The target items' TF-IDF vectors act as the input space, and the explicit ratings act as continuous target variables.
- **Optimization:** The models are optimized independently using a regularized Mean Squared Error (MSE) loss, with an L_2 penalty applied to the user's weight vector to prevent overfitting on sparse histories.
- **Inference:** Recommendation lists are generated using highly vectorized matrix multiplication between the pre-computed item feature matrix and the target user's learned weight vector, masking historical interactions before top- K extraction.

3.1.3. Data Preprocessing

The preprocessing stage extracts and structures the information necessary to represent items and interactions in the MovieLens-1M dataset [8]. In this pure content-based approach, the primary item features rely entirely on movie genres.

Let $\mathcal{I}$ be the set of movies. Each movie $i \in \mathcal{I}$ is associated with a string of genres separated by a delimiter (e.g., "Action—Adventure—Sci-Fi"). These strings are parsed into variable-length lists. A *MultiLabelBinarizer* is then fit to the entire corpus of genres to create a binary multi-hot matrix, where each column corresponds to a unique genre $g \in \mathcal{G}$.

To emphasize rare and highly informative genres while discounting overly common ones (such as "Drama" or "Comedy"), a Term Frequency-Inverse Document Frequency (TF-IDF) transformation, followed by L_2 normalization, is applied to the binary matrix [4]. The TF-IDF weight for a genre term t in movie document d is computed as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{1 + |\mathcal{I}|}{1 + \text{DF}(t)} \right) + 1, \quad (1)$$

where $\text{DF}(t)$ is the document frequency (the number of movies containing genre t). This transformation results in a static, continuous feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{I}| \times |\mathcal{G}|}$, where each movie is represented by a smooth, normalized vector $\mathbf{x}_i$.

For user-item interactions, raw user IDs and movie IDs are mapped to contiguous 0-based integer indices to facilitate array indexing during the isolated, per-user training loops.

3.1.4. Architecture

The architecture of this content-based recommender system is fundamentally centered on a decoupled, linear modeling approach.

Item Feature Layer. Items are entirely represented by the static $\mathbf{X}$ matrix generated during preprocessing. Unlike embedding layers in collaborative filtering, these vectors are frozen and provide an explicit, human-readable coordinate space where each dimension strictly corresponds to a specific movie genre.

User Profile Modeling. Instead of learning a single global model with shared parameters across all users, the architecture instantiates an independent regression model for each user $u \in \mathcal{U}$. The user’s profile is entirely parameterized by a learnable weight vector $\mathbf{w}_u \in \mathbb{R}^{|\mathcal{G}|}$ and a bias scalar $b_u \in \mathbb{R}$. The weight vector $\mathbf{w}_u$ serves as a direct proxy for the user’s affinity toward each genre.

Prediction Mechanism. The architecture consists of a single linear transformation mapping the target movie’s feature vector directly to a scalar rating prediction. The predicted rating $\hat{r}_{u,i}$ for a target movie i by user u is given by the inner product:

$$\hat{r}_{u,i} = \mathbf{w}_u^\top \mathbf{x}_i + b_u \quad (2)$$

Because it relies entirely on manually engineered TF-IDF genre features, the model’s capacity to capture complex semantics is bounded by the quality of this input space. However, this independent linear architecture offers extreme computational efficiency and explicit interpretability.

3.1.5. Training Objective

The model optimizes for explicit rating prediction error. For a given user u , let $\mathcal{I}_u$ denote the subset of movies rated by u , and $r_{u,i}$ denote the true observed rating.

Since the models are decoupled, the training objective is independently formulated for each user to find the optimal parameters $\mathbf{w}_u$ and b_u that minimize the regularized Mean Squared Error (MSE), effectively solving a Ridge Regression problem:

$$\mathcal{L}_u(\mathbf{w}_u, b_u) = \sum_{i \in \mathcal{I}_u} \left(r_{u,i} - (\mathbf{w}_u^\top \mathbf{x}_i + b_u) \right)^2 + \alpha \|\mathbf{w}_u\|_2^2 \quad (3)$$

where $\alpha > 0$ is the L_2 regularization hyperparameter.

The L_2 penalty term $\alpha \|\mathbf{w}_u\|_2^2$ is critical for systemic stability. Because users often have highly sparse interaction histories (e.g., having rated only a handful of movies), an unregularized model would severely overfit to the features of those few items. By shrinking the learned weights towards zero, the regularization ensures generalizability. Because this objective is strictly convex and independent across users, the optimization avoids iterative gradient descent entirely, relying instead on highly efficient closed-form linear algebraic solvers.

3.1.6. Inference

During the inference phase, the objective is to generate a ranked list of Top- K items for a target user u . The inference procedure is highly vectorized, leveraging the static nature of the item features.

1. **Parameter Retrieval:** The optimal learned weight vector $\mathbf{w}_u \in \mathbb{R}^{|\mathcal{G}|}$ and bias b_u for the target user are retrieved from the model state.
2. **Feature Matrix Loading:** The full, pre-computed item feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{I}| \times |\mathcal{G}|}$ is accessed.
3. **Affinity Scoring:** The predicted ratings over the entire item catalog are calculated simultaneously via matrix-vector multiplication:

$$\hat{\mathbf{r}}_u = \mathbf{X} \mathbf{w}_u + b_u \quad (4)$$

where $\hat{\mathbf{r}}_u \in \mathbb{R}^{|\mathcal{I}|}$.

4. **Interaction Masking:** To ensure the system recommends novel content, the predicted ratings for items the user has already explicitly interacted with in the training set are masked by setting their scores to $-\infty$.
5. **Top-K Retrieval:** The indices corresponding to the highest K values in the masked prediction vector $\hat{\mathbf{r}}_u$ are extracted.
6. **Metadata Integration:** The item indices are mapped back to their original Movie IDs to present the final recommendation slate to the user.

3.2. Memory-Based Collaborative Filtering

3.2.1. Overview

Memory-Based Collaborative Filtering (CF) is a heuristic-driven recommendation algorithm that operates directly on the historical user-item interaction matrix. The core assumption of this architecture is that users who agreed in their evaluations in the past will agree in the future (User-Based CF), or that users will prefer items that are structurally similar to those they have liked historically (Item-Based CF) [2].

In this architecture, the entire explicit rating matrix serves directly as the computational foundation. Recommendations are generated dynamically by identifying a local neighborhood of similar users or items using vector-space distance metrics, and then computing a weighted average of their explicit ratings. This non-parametric approach offers high interpretability and transparency, as every predicted score can be mathematically traced back to specific neighboring entities.

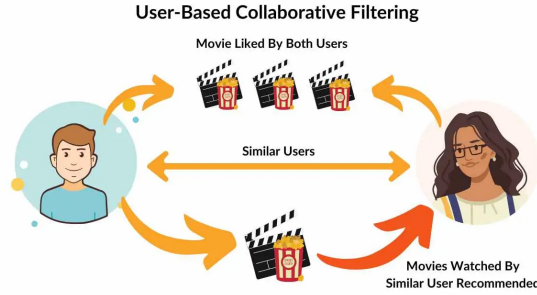


Figure 2: Core idea of Memory based CF.

3.2.2. Pipeline

The memory-based system is designed to support both User-Based and Item-Based configurations, structured through the following operational pipeline:

- **Data Preprocessing:** Raw MovieLens-1M data is ingested to construct a sparse utility matrix. To account for inherent rating biases (e.g., tough vs. generous raters), user-specific and item-specific means are calculated to generate a mean-centered, normalized utility matrix.
- **Similarity Computation:** The system computes the exact pairwise cosine similarity matrix across all target entities (users or items) based on the normalized utility matrix. The diagonal is masked to prevent self-similarity, and the resulting structure is cached.
- **Inference:** For a target user-item pair, the system dynamically retrieves the K -nearest neighbors from the pre-computed similarity matrix and executes a weighted aggregation to produce a final prediction score.
- **Evaluation:** The system ranks candidate items based on these predicted scores and evaluates performance using metrics such as Precision, Recall, NDCG, MRR, and RMSE.

3.2.3. Data Preprocessing

The preprocessing stage transforms raw MovieLens-1M interactions into sparse, normalized utility structures optimized for rapid algebraic similarity calculations. Let $\mathcal{U}$ be the set of users and $\mathcal{I}$ be the set of items. The system constructs a sparse utility matrix $R \in \mathbb{R}^{|\mathcal{U}| \times |\mathcal{I}|}$, where $R_{u,i}$ is the explicit rating given by user u to item i .

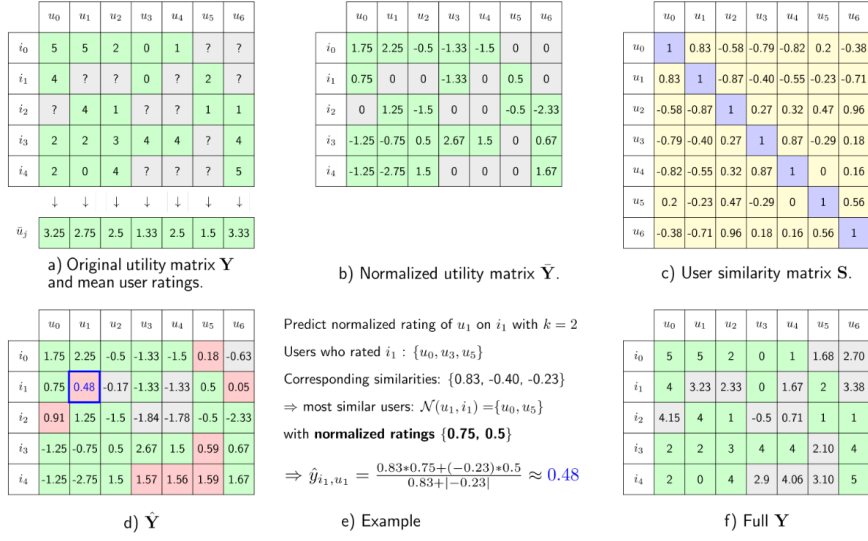


Figure 3: Memory-based CF process.

A critical vulnerability of raw distance metrics in rating matrices is the failure to account for inherent rating biases (e.g., A tough user who always rates below 3 stars is different from a generous one who often rates 5 stars). To resolve this, the system computes the mean rating μ_u for every user u and the mean rating μ_i for every item i .

Depending on the operational mode, the matrix is mean-centered. For User-Based CF:

$$\tilde{R}_{u,i} = R_{u,i} - \mu_u \quad (5)$$

For Item-Based CF:

$$\tilde{R}_{u,i} = R_{u,i} - \mu_i \quad (6)$$

These normalized ratings form the normalized sparse utility matrix $\tilde{\mathbf{R}}$. Mean-centering mathematically transforms the standard Cosine Similarity metric into the Pearson Correlation Coefficient, isolating the true affinity signal from baseline rating biases [22].

3.2.4. Architecture

The architecture of this memory-based recommender system is centered on vector-space similarity and localized aggregation. The system supports two dual architectures: User-Based and Item-Based CF. The fundamental components are structured as follows:

Vector Representation. Entities are represented directly by their interaction vectors extracted from the normalized sparse utility matrix $\tilde{\mathbf{R}}$. In the User-Based paradigm, each user $u \in \mathcal{U}$ is represented by a high-dimensional vector $\tilde{\mathbf{r}}_u \in \mathbb{R}^{|\mathcal{I}|}$ denoting their normalized ratings across all items. Conversely, in the Item-Based paradigm, each item $i \in \mathcal{I}$ is represented by a vector $\tilde{\mathbf{c}}_i \in \mathbb{R}^{|\mathcal{U}|}$ capturing the normalized ratings it received from all users.

Similarity Computation. The core mechanism for establishing relationships is the computation of symmetric cosine similarity between the normalized representation vectors. For two items i and j in the Item-Based architecture, the similarity is defined as:

$$\text{sim}(i, j) = \frac{\tilde{\mathbf{c}}_i \cdot \tilde{\mathbf{c}}_j}{\|\tilde{\mathbf{c}}_i\| \|\tilde{\mathbf{c}}_j\|} \quad (7)$$

Similarly, for two users u and v in the User-Based architecture:

$$\text{sim}(u, v) = \frac{\tilde{\mathbf{r}}_u \cdot \tilde{\mathbf{r}}_v}{\|\tilde{\mathbf{r}}_u\| \|\tilde{\mathbf{r}}_v\|} \quad (8)$$

This computation results in a dense global similarity matrix S . To ensure strict logical consistency during neighborhood retrieval, the diagonal of S is explicitly masked to $-\infty$ ($S_{k,k} = -\infty$), effectively removing self-referential relationships.

Prediction Aggregation. The final phase of the architecture is the dynamic aggregation of neighbor ratings to produce a scalar prediction score $\hat{r}_{u,i}$ for a specific user u and item i .

In the Item-Based architecture, the system identifies the subset of items $\mathcal{N}_u$ that user u has previously rated. It then extracts the Top- K items from $\mathcal{N}_u$ that exhibit the highest similarity $\text{sim}(i, j)$ to the target item i . The predicted rating is computed as the item's baseline mean adjusted by the weighted sum of the user's historical normalized ratings:

$$\hat{r}_{u,i} = \mu_i + \frac{\sum_{j \in \text{Top}K(\mathcal{N}_u)} \text{sim}(i, j) \times \tilde{R}_{u,j}}{\sum_{j \in \text{Top}K(\mathcal{N}_u)} |\text{sim}(i, j)|} \quad (9)$$

In the User-Based architecture, the system isolates the subset of users $\mathcal{N}_i$ who have rated item i . It selects the Top- K users most similar to the target user u , and computes the prediction as:

$$\hat{r}_{u,i} = \mu_u + \frac{\sum_{v \in \text{Top}K(\mathcal{N}_i)} \text{sim}(u, v) \times \tilde{R}_{v,i}}{\sum_{v \in \text{Top}K(\mathcal{N}_i)} |\text{sim}(u, v)|} \quad (10)$$

If the intersection of rated neighbors is empty (e.g., severe cold-start scenario), the architecture defaults to a fallback mechanism, predicting the global item mean μ_i or the absolute global mean μ_{global} to ensure a valid numerical output. Finally, all predicted scores are explicitly clipped to the valid $[1.0, 5.0]$ range.

Based on the mathematical formulas, memory-based filtering is fundamentally a weighted k-nearest neighbors (KNN) algorithm.

3.2.5. Training Objective

The "training objective" of the memory-based architecture is exclusively computational caching rather than empirical error minimization. The goal is to comprehensively compute the exact global similarity matrix S over the normalized utility matrix $\tilde{R}$, and store this structure persistently.

For a dataset containing $|\mathcal{U}|$ users and $|\mathcal{I}|$ items, dynamically computing similarities for every inference request would result in an intractable $O(|\mathcal{U}| \times |\mathcal{I}|)$ runtime complexity. By fully precomputing the symmetric cosine similarity matrix during the training phase, the system effectively shifts the computational burden. This caching strategy ensures that during inference, the identification of Top- K nearest neighbors requires only an optimized $O(1)$ algebraic lookup per candidate, enabling the system to scale efficiently.

3.2.6. Inference

During the inference phase, the objective is to generate a ranked recommendation list or predict specific rating scores for a target user u over unseen candidate items.

1. **Target Isolation:** The system extracts the explicit interaction history of the target user u directly from the utility matrix, identifying all items the user has previously rated.

2. **Candidate Generation:** To ensure the system recommends strictly novel content, the historically rated items are masked out. The remaining unrated items in the catalog form the candidate set.
3. **On-Demand Prediction (Lazy-Evaluation):** For the target candidate items, the system invokes the prediction aggregation layer. Instead of predicting the entire $u \times \mathcal{I}$ matrix space, the system dynamically retrieves only the necessary similarity vectors from the pre-computed matrix S .
4. **Top-K Neighbor Aggregation:** The system slices the similarity vectors to match the user’s historical items (or interacting users), sorts them to extract the Top- K highest values, and computes the weighted mathematical average to predict the final rating..

3.3. Hybrid Recommender System

3.3.1. Overview

The hybrid recommender system combines the complementary strengths of memory-based collaborative filtering and content-based filtering. Memory-based collaborative filtering captures collective user–item interaction patterns, while content-based filtering uses item metadata to estimate whether a movie matches a user’s observed preferences. Each approach has different failure modes: collaborative filtering can struggle with sparse histories and cold-start cases, whereas content-based filtering can over-specialize around the features already observed for a user. A hybrid approach is therefore used to blend their prediction signals into a single recommendation score.

In this project, the hybrid model is implemented as a weighted hybrid layer over three independently trained component models: item-based collaborative filtering, user-based collaborative filtering, and content-based filtering. For a user u and candidate item i , the component predictions are denoted by $\hat{r}_{u,i}^{\text{item}}$, $\hat{r}_{u,i}^{\text{user}}$, and $\hat{r}_{u,i}^{\text{content}}$, respectively. The hybrid prediction is computed as

$$\hat{r}_{u,i}^{\text{hybrid}} = \alpha \hat{r}_{u,i}^{\text{item}} + \beta \hat{r}_{u,i}^{\text{user}} + \gamma \hat{r}_{u,i}^{\text{content}}, \quad (11)$$

where α, β , and γ are non-negative blending weights satisfying

$$\alpha + \beta + \gamma = 1, \quad \gamma = 1 - \alpha - \beta. \quad (12)$$

This formulation allows the system to compensate for the weaknesses of any single component. For example, when collaborative filtering signals are unreliable for sparse users, the content-based score can still provide a meaningful estimate based on movie genres. Conversely, when genre information is too coarse, collaborative filtering can capture behavioral signals from similar users or similar items.

3.3.2. Pipeline

The proposed system follows a four-stage pipeline for the hybrid recommendation layer:

- **Data processing:** Raw data are transformed into a shared indexing scheme. This stage resolves the mismatch between movies appearing in the ratings file and movies appearing in the metadata file, constructs genre-based item features for the content-based model, and creates train, validation, and test splits.
- **Component model training:** The item-based collaborative filtering, user-based collaborative filtering, and content-based models are trained independently. Each model produces a prediction score for user–item pairs using its own representation and objective.
- **Hybrid weight tuning:** The hybrid layer selects the blending weights α and β using grid search on the validation set. Since γ is determined by Equation (12), only two independent weights need to be tuned.
- **Inference:** At serving time, the tuned weights are retrieved for the target user’s group, the three component models generate scores for candidate movies, and the weighted hybrid score in Equation (11) is used to rank the final recommendations.

3.3.3. Data Processing

A key preprocessing issue is that the movie catalogue and the rating matrix do not contain exactly the same set of movies. The metadata file contains 3,883 movies, but the ratings file contains ratings for only 3,706 of them. In addition, the original movie IDs are not contiguous, so directly using raw IDs as matrix indices would introduce gaps and inconsistent indexing across component models.

This issue is especially important for the hybrid architecture because the model blends prediction matrices produced by different pipelines. Collaborative filtering builds its utility matrix from the ratings data, while content-based filtering builds its feature matrix from movie metadata. If the two pipelines assign different meanings to the same matrix column, the hybrid layer may silently combine scores for different movies. To avoid this, the system keeps only movies with at least one observed rating and constructs a single universal movie-index mapping shared by all component models.

For the content-based component, movie genre strings are parsed into multi-hot vectors over the genre vocabulary and then transformed into TF-IDF features. This produces an item feature matrix

$$\mathbf{X} \in \mathbb{R}^{|I| \times |\mathcal{G}|}, \quad (13)$$

where $|I| = 3,706$ is the number of reindexed rated movies and $|\mathcal{G}|$ is the number of genre categories. These item features are used by the content-based model, while the same item indices are used by the collaborative filtering models.

The hybrid layer also requires a validation set in addition to train and test sets. The train set is used to fit the three component models, the validation set is used to tune the hybrid weights, and the test set is reserved for final evaluation. Two split strategies are used. Under leave-one-out splitting, the most recent rating of each user is assigned to the test set, the second-most-recent rating is assigned to the validation set, and the remaining earlier ratings are used for training. Under the 80–10–10 split, each user’s ratings are divided into 80% training, 10% validation, and 10% testing data, ensuring that each user is represented proportionally in all splits.

3.3.4. Architecture

The hybrid architecture is a second-stage model built on top of three component recommenders. The item-based collaborative filtering component predicts ratings by comparing candidate items against items the target user has already rated, using item-mean-normalized cosine similarity. The user-based collaborative filtering component predicts ratings by finding users with similar rating patterns, using user-mean-normalized cosine similarity. The content-based filtering component predicts ratings using a per-user Ridge regression model over the TF-IDF genre features described in Equation (13).

Let the three full prediction matrices be

$$\hat{\mathbf{R}}^{\text{item}}, \quad \hat{\mathbf{R}}^{\text{user}}, \quad \hat{\mathbf{R}}^{\text{content}} \in \mathbb{R}^{|\mathcal{U}| \times |I|}. \quad (14)$$

The hybrid prediction matrix is then

$$\hat{\mathbf{R}}^{\text{hybrid}} = \alpha \hat{\mathbf{R}}^{\text{item}} + \beta \hat{\mathbf{R}}^{\text{user}} + (1 - \alpha - \beta) \hat{\mathbf{R}}^{\text{content}}. \quad (15)$$

Although memory-based predictions can be generated on demand, full prediction matrices are precomputed once during offline tuning. This avoids repeatedly recomputing expensive nearest-neighbor predictions for every validation user, candidate item, and weight combination. The on-demand prediction functions are still retained for the final inference procedure, where only one user’s candidate set needs to be scored.

Cold and Warm User Switching. The hybrid architecture also includes a simple switching mechanism for cold-start adaptation. Users with fewer than 30 training ratings are classified as cold users, while users with at least 30 training

ratings are classified as warm users. Cold users have limited interaction history, so collaborative filtering signals are less reliable and the content-based component is expected to receive more weight. Warm users provide richer interaction histories, allowing collaborative filtering to contribute more strongly.

Instead of learning a single global pair of weights, the model therefore maintains two independent weight pairs:

$$(\alpha_{\text{cold}}, \beta_{\text{cold}}), \quad (\alpha_{\text{warm}}, \beta_{\text{warm}}). \quad (16)$$

During both tuning and inference, each user is routed to the corresponding weight pair based on the number of ratings they contributed to the training set.

3.3.5. Training Objective

The hybrid layer has only two independent trainable parameters, α and β , because γ is determined by Equation (12). These weights are selected by grid search on the validation set using a ranking-based objective. The search uses a step size of 0.05:

$$\alpha \in \{0.00, 0.05, \dots, 1.00\}, \quad \beta \in \{0.00, 0.05, \dots, 1.00 - \alpha\}. \quad (17)$$

The constraint $\alpha + \beta \leq 1$ ensures that all three weights are non-negative. The number of valid weight combinations is

$$\sum_{k=0}^{20} (21 - k) = \frac{21 \times 22}{2} = 231. \quad (18)$$

Thus, the triangular search space reduces the number of combinations from the naive $21 \times 21 = 441$ square grid to 231 valid blends.

For a fixed pair (α, β) , the validation score for user u and candidate item i is

$$s_{u,i} = \alpha \hat{R}_{u,i}^{\text{item}} + \beta \hat{R}_{u,i}^{\text{user}} + (1 - \alpha - \beta) \hat{R}_{u,i}^{\text{content}}. \quad (19)$$

Candidate movies are ranked in descending order of $s_{u,i}$, and the weights are selected to maximize mean NDCG@ K on the validation users:

$$\text{NDCG}@K = \frac{1}{|\mathcal{U}_{\text{val}}|} \sum_{u \in \mathcal{U}_{\text{val}}} \frac{\text{DCG}_u@K}{\text{IDCG}_u@K}, \quad (20)$$

where

$$\text{DCG}_u@K = \sum_{k=1}^K \frac{\mathbb{I}[\text{item at rank } k \text{ is relevant}]}{\log_2(k+1)}. \quad (21)$$

Here, $\text{IDCG}_u@K$ is the maximum achievable discounted cumulative gain for user u . NDCG is used instead of RMSE because the hybrid recommender is evaluated as a top- K ranking system. RMSE measures numerical rating accuracy, whereas NDCG measures whether relevant items are ranked near the top of the recommendation list.

The full grid search is performed separately for cold and warm users, producing the two weight pairs in Equation (16). This allows the hybrid system to assign different importance to collaborative and content-based signals depending on the amount of available user history.

3.3.6. Inference Procedure

During inference, the system first determines whether the target user is cold or warm using the 30-rating threshold from the training set. The corresponding tuned weight pair is then retrieved. For the target user's candidate movies, the item-based collaborative filtering model, user-based collaborative filtering model, and content-based model each produce prediction scores. These scores are blended using Equation (11). Movies already rated by the user are masked out, and the remaining items are sorted by the hybrid score to return the final Top- K recommendation list.

This design separates expensive offline tuning from lightweight online serving. Full prediction matrices are useful for efficient validation-time weight search, while on-demand scoring is used during inference to simulate a practical recommendation backend that responds to a single user request without recomputing the entire user-item matrix.

3.4. Matrix Factorization

3.4.1. Overview

Matrix Factorization (MF) is one of the most influential model-based approaches for collaborative filtering. Instead of directly comparing users or items based on observed interactions, MF assumes that both users and items can be represented in a shared latent feature space learned from historical rating data [5].

The key idea is that each user is associated with a latent preference vector, while each item is represented by a latent characteristic vector. These latent factors capture hidden properties that explain user-item interactions, such as genre preferences, thematic interests, or other abstract patterns that are not explicitly provided in the dataset.

Given a user latent vector $\mathbf{p}_u$ and an item latent vector $\mathbf{q}_i$, the predicted preference of user u for item i is modeled through their inner product. Intuitively, a higher compatibility between the two vectors indicates a greater likelihood that the user will appreciate the item.

MF approximates the sparse user-item rating matrix $\mathbf{R}$ as the product of two low-rank matrices:

$$\mathbf{R} \approx \mathbf{P}\mathbf{Q}^T, \quad (22)$$

where $\mathbf{P}$ contains the latent representations of users and $\mathbf{Q}$ contains the latent representations of items.

By learning these latent representations directly from data, Matrix Factorization can uncover hidden relationships between users and items, alleviate data sparsity, and provide more accurate recommendations than traditional memory-based collaborative filtering methods

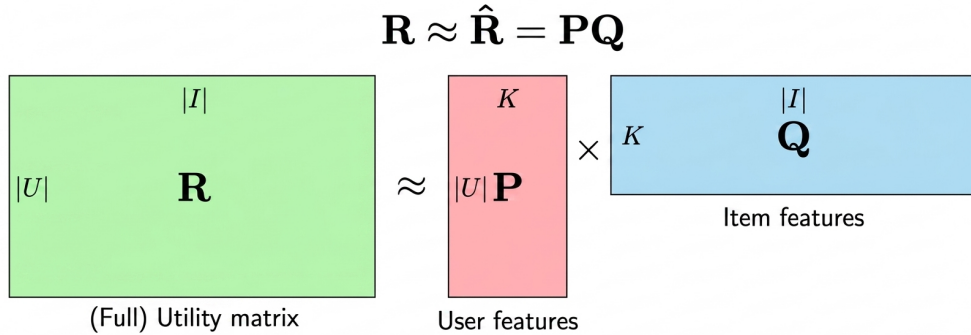


Figure 4: Matrix Factorization. Utility matrix $\mathbf{R}$ is decomposed into the product of two low-rank matrices $\mathbf{P}$ and $\mathbf{Q}$

3.4.2. Pipeline

The proposed system employs a highly modular and robust pipeline designed to train and evaluate the latent factor model at scale:

- **Data Preprocessing:** Raw MovieLens-1M data is ingested, and explicit identifier strings (user IDs and movie IDs) are mapped to contiguous integer indices. This establishes a mathematically sound domain for matrix indexing and embedding lookups.
- **Model Initialization:** The parameter space—comprising the user/item latent matrices and bias vectors—is instantiated. Latent factors are drawn from a scaled normal distribution to ensure numerical stability during the early stages of optimization.

- **Optimization:** The system iterates over the training interactions using Stochastic Gradient Descent (SGD). For each empirical observation, the system computes the predictive error and applies backpropagation to update the specific factors and biases involved, heavily regularized by an L_2 penalty to prevent overfitting.
- **Inference:** Recommendation scores are derived dynamically by executing dot products between the learned user and item embeddings and adding the corresponding bias scalars. Unseen items are masked, and the final predictions are ranked to extract the Top- K items for the target user.

3.4.3. Data Preprocessing

The data preprocessing stage focuses on structuring the raw historical interactions into a format optimized for parameterized machine learning. The MovieLens-1M dataset consists of explicit user-item interactions represented as quadruplets: (user_id, item_id, rating, timestamp).

Let $\mathcal{U}$ define the universal set of users and $\mathcal{I}$ define the universal set of items. The raw, non-sequential alphanumeric IDs are computationally inefficient for array operations. Consequently, the preprocessing stage extracts the unique elements of $\mathcal{U}$ and $\mathcal{I}$ and maps them into two distinct dictionaries, ensuring that all users and items are represented by dense, 0-indexed integers. This guarantees that every user $u \in \{0, 1, \dots, |\mathcal{U}| - 1\}$ and item $i \in \{0, 1, \dots, |\mathcal{I}| - 1\}$ corresponds directly to specific rows in the parameterized weight matrices.

For temporal or randomized splits, the preprocessed index mappings are preserved. The resultant dataset is isolated into a structured numerical tensor containing only the [user_idx, item_idx, rating] dimensions, ready to be iterated over during the stochastic optimization cycles.

3.4.4. Architecture

The Matrix Factorization architecture is designed around parametric representation and continuous algebraic projection. The fundamental structural components define how entities are mathematically represented and how these representations combine to form a predictive signal.

Latent Representation. The architecture maps every user and item to a joint latent feature space of dimensionality K . Each user u is associated with a parameterized latent vector $\mathbf{p}_u \in \mathbb{R}^K$, forming the user embedding matrix $P \in \mathbb{R}^{|\mathcal{U}| \times K}$. Similarly, each item i is associated with a latent vector $\mathbf{q}_i \in \mathbb{R}^K$, forming the item embedding matrix $Q \in \mathbb{R}^{|\mathcal{I}| \times K}$. To initiate the symmetry breaking required for effective gradient-based optimization, these matrices are initialized with values sampled from a standard normal distribution, specifically $\mathcal{N}(0, 1/K)$.

Bias Formulation. Because observed ratings are heavily influenced by effects independent of latent user-item interactions, the architecture integrates independent bias scalars. It learns a global average rating $\mu \in \mathbb{R}$ to establish a systemic baseline. Furthermore, it parameterizes a distinct bias term $b_u \in \mathbb{R}$ for every user to model their general tendency to rate higher or lower than the systemic average, and a bias term $b_i \in \mathbb{R}$ for every item to model its inherent popularity or critical reception. These parameters form the bias vectors $\mathbf{b}_u \in \mathbb{R}^{|\mathcal{U}|}$ and $\mathbf{b}_i \in \mathbb{R}^{|\mathcal{I}|}$, initialized entirely to zero.

Prediction Synthesis Mechanism. The core predictive capability of the architecture relies on synthesizing these learned components into a singular continuous scalar. The predicted explicit rating $\hat{r}_{u,i}$ for a specific user u and target item i is formulated mathematically as:

$$\hat{r}_{u,i} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i \quad (23)$$

This formulation dictates that the relevance score is a composition of the global dataset bias, the structural biases of the specific user and item, and the deep interaction signal captured by the inner product $\mathbf{p}_u^\top \mathbf{q}_i$, which measures the alignment between the user's preferences and the item's features within the K -dimensional latent space. If the system

encounters a completely novel user or item (the cold-start regime), the architecture possesses a built-in fallback logic, safely defaulting the prediction to the global mean μ .

3.4.5. Training Objective

The architectural parameters $\{P, Q, \mathbf{b}_u, \mathbf{b}_i, \mu\}$ are optimized through an explicit empirical error minimization strategy. For a given training set $\mathcal{K}$ containing known ratings $r_{u,i}$, the system minimizes the regularized squared error function:

$$\mathcal{L} = \sum_{(u,i) \in \mathcal{K}} (r_{u,i} - \hat{r}_{u,i})^2 + \lambda (\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2) \quad (24)$$

where λ controls the magnitude of the L_2 regularization penalty, crucial for constraining parameter explosion and preventing the model from severely overfitting to the highly sparse observed interactions.

Optimization is executed iteratively using Stochastic Gradient Descent (SGD). For every individual observation $(u, i, r_{u,i})$, the prediction error is calculated as $e_{u,i} = r_{u,i} - \hat{r}_{u,i}$. The parameters are then updated simultaneously in the direction of the negative gradient, modulated by a learning rate γ :

$$\mu \leftarrow \mu + \gamma \cdot e_{u,i} \quad (25)$$

$$b_u \leftarrow b_u + \gamma \cdot (e_{u,i} - \lambda b_u) \quad (26)$$

$$b_i \leftarrow b_i + \gamma \cdot (e_{u,i} - \lambda b_i) \quad (27)$$

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \gamma \cdot (e_{u,i} \mathbf{q}_i - \lambda \mathbf{p}_u) \quad (28)$$

$$\mathbf{q}_i \leftarrow \mathbf{q}_i + \gamma \cdot (e_{u,i} \mathbf{p}_u - \lambda \mathbf{q}_i) \quad (29)$$

This continuous backpropagation cycle incrementally refines the latent representations, systematically driving the parameter space toward an optimal configuration capable of highly generalized inference.

3.4.6. Inference

During the operational inference phase, the model is queried to generate explicit numeric predictions or highly ranked recommendation slates for target users over unobserved candidates. The inference pipeline operates systematically as follows:

1. **Parameter Retrieval:** The optimal, fully converged states of the latent matrices P, Q and the scalar biases $\mathbf{b}_u, \mathbf{b}_i, \mu$ are extracted from persistent storage into active memory.
2. **Target Slicing:** For a target user u , their specific latent vector $\mathbf{p}_u$ and scalar bias b_u are isolated via $O(1)$ index lookup. The historical interaction matrix is scanned to identify and mask the catalog of items the user has already explicitly rated.
3. **Vectorized Scoring:** The system predicts the affinity scores for all candidate items $i \in \mathcal{I}$ simultaneously through highly optimized linear algebra. The predicted vector $\hat{\mathbf{r}}_u \in \mathbb{R}^{|\mathcal{I}|}$ is computed via the broadcasted addition of the global biases and the matrix-vector multiplication $Q\mathbf{p}_u$.
4. **Top-K Retrieval:** The indices corresponding to the maximum K values within the newly computed prediction vector $\hat{\mathbf{r}}_u$ are extracted.

3.5. Neural Collaborative Filtering (NCF)

3.5.1. Overview

Neural Collaborative Filtering (NCF) is a generalized deep learning framework designed to replace the simple inner product of traditional Matrix Factorization with a non-linear neural network architecture [23]. While classical latent

factor models are inherently limited by their linear nature, NCF leverages the representation capabilities of deep neural networks to model highly complex, non-linear user-item interactions.

The core philosophy of NCF is to capture distinct levels of feature interactions by integrating two parallel, structurally independent pathways: a Generalized Matrix Factorization (GMF) branch designed to capture low-level, linear collaborative signals, and a Multi-Layer Perceptron (MLP) branch engineered to extract high-level, non-linear interaction patterns. This dual-pathway approach empowers the system to flexibly approximate any continuous interaction function, resolving the representational bottleneck of pure linear models and delivering highly personalized recommendation ranking.

3.5.2. Pipeline

The implementation operates via an end-to-end deep learning pipeline, optimized for large-scale PyTorch processing:

- **Data Preprocessing:** Explicit ratings from MovieLens-1M are converted into implicit feedback signals (binarized interactions) via an explicit threshold. The sequential data is safely partitioned into discrete training and testing sets utilizing a per-user randomized 80-20 protocol (or temporal leave-one-out for full-rank evaluation).
- **Negative Sampling:** Because implicit feedback lacks inherent negative examples, the pipeline dynamically injects unobserved user-item pairs into the training batches (negative sampling) at every epoch to formulate a binary classification problem.
- **Model Initialization & Pre-training:** The architecture initializes separate embedding tables for the GMF and MLP branches. To facilitate optimization of the highly non-convex objective, the system supports a pre-training mechanism, initializing the NeuMF model using weights systematically loaded and fused from independently trained GMF and MLP baseline networks.
- **Optimization:** The system trains the parameterized network using mini-batch gradient descent. For from-scratch training NeuMF, GMF and MLP, it utilizes the Adam optimizer, while for pre-trained fine-tuning NeuMF, it applies Stochastic Gradient Descent (SGD) with momentum to carefully preserve the structural integrity of the pre-trained embeddings.
- **Inference:** The network executes a highly parallelized forward pass on candidate items, computing normalized interaction probabilities via a final Sigmoid activation. These scalar probabilities dictate the final Top- K ranking slate.

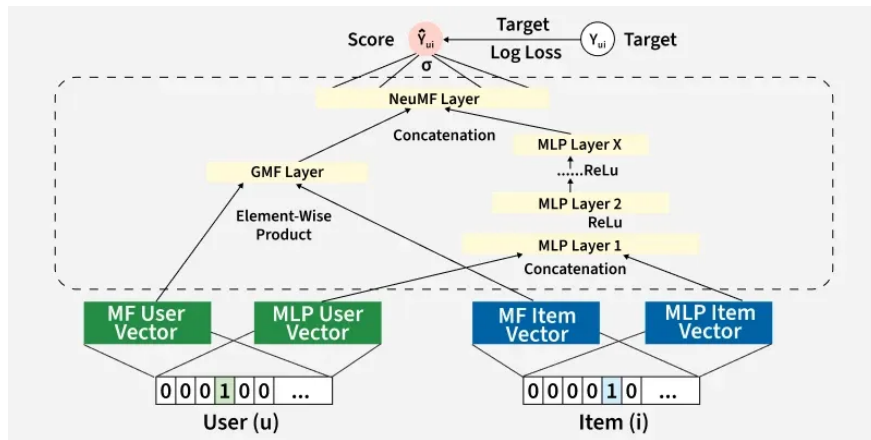


Figure 5: Neural Collaborative Filtering framework

3.5.3. Data Preprocessing

To adapt the explicitly rated MovieLens-1M dataset for the binary classification objective of NCF, the preprocessing stage implements an implicit thresholding transformation. The original ratings in the range $[1, 5]$ reflect varying levels of user preference and are therefore transformed into binary interaction signals. To normalize the learning target, the system applies an explicit filter: ratings greater than or equal to 3.0 are retained and projected as positive interactions ($y_{u,i} = 1$), denoting explicit user affinity. Interactions below this threshold are discarded to prevent injecting contradictory or weak signals into the positive manifold.

For rigorous evaluation and to absolutely prevent data leakage, the positive interactions are structurally partitioned utilizing a strictly per-user randomized 80-20 split. The partitioned tensors are subsequently cached to disk. To facilitate PyTorch embedding table allocations, the explicitly provided 1-indexed MovieLens integers (which contain inherent structural gaps) are dynamically compressed into dense, contiguous 0-indexed integer vectors. This structural mapping ensures complete memory efficiency and strictly prevents out-of-bounds index exceptions during backpropagation lookups.

3.5.4. Deep Architecture

The Neural Collaborative Filtering (NeuMF) architecture is a multi-branch neural network designed to capture heterogeneous collaborative signals. The architecture mathematically unifies two independent paradigms into a single, cohesive forward pass.

Embedding Representation Layer. Because the linear (GMF) and non-linear (MLP) pathways optimize fundamentally different geometric spaces, the architecture allocates completely independent embedding tables for each branch. For a given user u and item i , the GMF branch retrieves the latent vectors $\mathbf{p}_u^G \in \mathbb{R}^{K_1}$ and $\mathbf{q}_i^G \in \mathbb{R}^{K_1}$ from the GMF embedding tables. Concurrently, the MLP branch retrieves distinct latent vectors $\mathbf{p}_u^M \in \mathbb{R}^{K_2}$ and $\mathbf{q}_i^M \in \mathbb{R}^{K_2}$ from the independent MLP embedding tables, where K_1 and K_2 specify the distinct latent dimensionalities of the two branches.

Generalized Matrix Factorization (GMF) Branch. The GMF branch models explicit low-rank linear interactions between users and items and can be viewed as a generalized extension of traditional Matrix Factorization (MF).

In classical Matrix Factorization, the interaction score between a user and an item is estimated using the inner product of their latent vectors:

$$\hat{r}_{u,i} = \mathbf{p}_u^T \mathbf{q}_i \quad (30)$$

where $\mathbf{p}_u$ and $\mathbf{q}_i$ denote the latent representations of user u and item i , respectively.

Although effective, this formulation implicitly assumes that all latent dimensions contribute equally to the final prediction. In practice, different latent factors may carry different levels of importance. GMF addresses this limitation by preserving the entire interaction vector and allowing the model to learn the relative importance of each latent dimension.

For a given user u and item i , the GMF branch retrieves:

$$\mathbf{p}_u^G \in \mathbb{R}^{K_1} \quad (31)$$

$$\mathbf{q}_i^G \in \mathbb{R}^{K_1} \quad (32)$$

The interaction representation is computed using the Hadamard product:

$$\phi^{GMF} = \mathbf{p}_u^G \odot \mathbf{q}_i^G \quad (33)$$

Unlike standard Matrix Factorization, the resulting interaction vector is preserved rather than immediately reduced to a scalar value. This allows subsequent layers to learn the contribution of each latent interaction independently.

Multi-Layer Perceptron (MLP) Branch. While the GMF branch focuses on modeling linear collaborative signals, the MLP branch is designed to capture complex non-linear relationships and higher-order feature interactions.

Instead of explicitly defining how user and item embeddings should interact, MLP learns a non-linear interaction function directly from data through a hierarchy of hidden layers.

The MLP branch maintains embedding tables independent from GMF:

$$\mathbf{p}_u^M \in \mathbb{R}^{K_2} \quad (34)$$

$$\mathbf{q}_i^M \in \mathbb{R}^{K_2} \quad (35)$$

The user and item embeddings are concatenated to form the input representation:

$$\mathbf{z}_0 = [\mathbf{p}_u^M; \mathbf{q}_i^M] \quad (36)$$

This vector is propagated through a deep feedforward neural network:

$$\mathbf{z}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{z}_0 + \mathbf{b}_1) \quad (37)$$

$$\vdots \quad (38)$$

$$\mathbf{z}_L = \text{ReLU}(\mathbf{W}_L \mathbf{z}_{L-1} + \mathbf{b}_L) \quad (39)$$

where $\mathbf{W}_l$ and $\mathbf{b}_l$ denote the trainable weight matrix and bias vector of the l -th hidden layer.

The final hidden representation is defined as:

$$\phi^{MLP} = \mathbf{z}_L \in \mathbb{R}^{L_{out}} \quad (40)$$

Unlike GMF, which relies on a predefined element-wise interaction, the MLP branch learns a flexible non-linear mapping

$$\phi^{MLP} = f(\mathbf{p}_u^M, \mathbf{q}_i^M) \quad (41)$$

where $f(\cdot)$ is represented by the deep neural network itself. This allows the model to capture higher-order user-item interaction patterns that cannot be effectively represented by traditional latent factor models.

NeuMF Fusion Layer. The final user-item representation is formed by concatenating the latent interaction vectors learned from the GMF and MLP branches. The resulting fused representation is subsequently fed into a fully connected output layer with a Sigmoid activation function, producing a normalized interaction probability in the range $[0,1]$:

$$\hat{y}_{u,i} = \sigma(\mathbf{h}^T [\boldsymbol{\phi}^{GMF}; \boldsymbol{\phi}^{MLP}]) \quad (42)$$

where $\mathbf{h} \in \mathbb{R}^{K_1+L_{out}}$ serves as the unified projection weight vector. If the network is initialized utilizing the pre-training methodology, the pre-trained weights from the independent GMF and MLP projection layers are concatenated and explicitly scaled by a hyperparameter $\alpha = 0.5$ before being assigned to $\mathbf{h}$, ensuring an equitable initial fusion of linear and non-linear logic.

3.5.5. Training Objective

Because NCF formulates recommendation as an implicit-feedback binary classification problem, each user-item pair is assigned a binary label:

$$y_{u,i} = \begin{cases} 1, & \text{if user } u \text{ has positive interaction with item } i \text{ (rating } \hat{r} = 3) \\ 0, & \text{otherwise} \end{cases} \quad (43)$$

The model is optimized using the Binary Cross-Entropy (BCE) loss function:

$$\mathcal{L} = - \sum_{(u,i)} [y_{u,i} \log(\hat{y}_{u,i}) + (1 - y_{u,i}) \log(1 - \hat{y}_{u,i})] \quad (44)$$

where:

- $y_{u,i}$ denotes the ground-truth interaction label.
- $\hat{y}_{u,i}$ denotes the interaction probability predicted by the model.

Binary Cross-Entropy is particularly suitable for implicit-feedback recommendation because the objective is to estimate interaction probabilities rather than explicit rating values. Furthermore, BCE naturally integrates with negative sampling, where randomly selected unobserved interactions are treated as negative examples during training.

The optimization landscape of NeuMF remains highly non-convex due to the deep neural architecture of the MLP branch. The complete parameter set is defined as

$$\Theta = \{P^G, Q^G, P^M, Q^M, \mathbf{W}_l, \mathbf{b}_l, \mathbf{h}\} \quad (45)$$

and is optimized through mini-batch gradient descent using either Adam (for training from scratch) or SGD with Momentum (for fine-tuning pre-trained models). The gradients are computed automatically through PyTorch's Autograd engine and propagated backward using standard backpropagation.

3.5.6. Inference

During the inference phase, the objective is to generate personalized recommendation slates for target users over all unseen candidate items utilizing the fully converged deep network. The procedure is optimized utilizing tensor broadcasting:

1. **Candidate Isolation:** For a specific target user u , the system structurally masks out all items present in the user’s explicit historical interaction dataset. The remaining subset defines the set of candidate items.
2. **Tensor Duplication:** The target user identifier is replicated through tensor broadcasting to create a uniform user-tensor $\mathbf{U} = [u, u, \dots, u]$ matching the exact dimension of the unobserved candidate item tensor $\mathbf{I} = [i_1, i_2, \dots, i_n]$.
3. **Parallel Forward Pass:** The user and item tensors are fed into the trained NeuMF network and subjected to a parallelized forward pass through the frozen NeuMF network. The network simultaneously queries the dual embedding tables, computes the GMF element-wise products, propagates through the deep MLP hidden layers, and synthesizes the outputs in the fusion layer.
4. **Probabilistic Scoring:** The output layer applies the final Sigmoid activation, mapping all entity interactions to a continuous scalar probability field $\hat{\mathbf{y}}_u \in [0, 1]^n$.
5. **Top-K Slate Generation:** The probability vector is sorted in descending order, and the target indices associated with the maximum K probabilistic values are extracted. These abstract indices are subsequently mapped back to the raw MovieLens metadata to formulate the final recommendation list presented to the user.

3.6. Two-Tower

3.6.1. Overview

A Two-Tower model, also called a dual-encoder retrieval model, learns two separate neural encoders that map a query object and a candidate object into the same latent space. In recommendation, the query is often a user, session, context, or source item, while the candidate is an item that may be retrieved. This design is related to earlier neural semantic matching models, where two inputs are projected into a shared representation space and compared by a simple similarity function [14]. It is also consistent with the retrieval stage of large-scale recommender systems, where a candidate generation model first narrows a large catalogue before a more expressive ranking model performs detailed scoring [13].

Formally, let x_q denote the query-side features and x_c denote the candidate-side features. The query tower f_θ and candidate tower g_ϕ produce embeddings

$$\mathbf{p}_q = \text{normalize}(f_\theta(x_q)), \quad \mathbf{q}_c = \text{normalize}(g_\phi(x_c)), \quad (46)$$

where both vectors lie in the same d -dimensional embedding space. The relevance score is then computed by a lightweight matching function such as an inner product or cosine similarity. Because the candidate embedding depends only on candidate-side features, all candidate vectors can be computed offline and searched efficiently at inference time using exact or approximate nearest-neighbor retrieval.

In this project, the generic query object is instantiated as a MovieLens user and the generic candidate object is instantiated as a movie. The query tower is therefore referred to as the User Tower, and the candidate tower is referred to as the Item Tower. The overall architecture is shown in Figure 6.

3.6.2. Pipeline

The proposed system follows a four-stage Two-Tower retrieval pipeline for movie recommendation:

- **Preprocessing:** Raw MovieLens data are transformed into model-ready inputs. This stage includes selecting positive interactions, encoding categorical user and movie features, converting movie genres into a lookup representation, and building fixed-length user history sequences from past interactions.
- **Two-Tower representation learning:** The processed data are passed into a Two-Tower neural architecture. The User Tower learns a dense user representation from user profile features and historical movie interactions, while the Item Tower learns a dense item representation from movie identifiers and genre information. The two

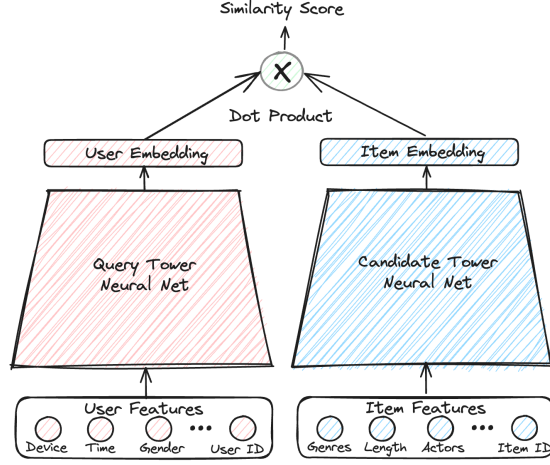


Figure 6: Overview of the Two-Tower recommender system architecture.

towers share the movie embedding layer, allowing movie representations to be learned consistently from both user histories and target items. Both towers project their outputs into the same 64-dimensional latent space and apply L2 normalization.

- **Similarity calculation:** After obtaining the user and item embeddings, the model computes their similarity using the dot product. Since both embeddings are L2-normalized, this dot product is equivalent to cosine similarity. During inference, embeddings for all candidate movies are precomputed, and recommendations are generated by ranking items according to their similarity scores with the target user vector. Previously interacted movies are masked out before producing the Top- K recommendation list.
- **Parameter update:** During training, the model uses an in-batch softmax loss. The matched user–item pair is treated as the positive example, while other items in the same batch serve as negative samples. The loss is then backpropagated to update the trainable parameters of the model, including the embedding layers, Transformer Encoder, and MLP layers. This optimization encourages relevant user–item pairs to be closer together in the shared embedding space.

3.6.3. Data Preprocessing

The preprocessing stage transforms the raw MovieLens-1M data into structured samples suitable for the Two-Tower retrieval model [8]. First, raw user IDs and movie IDs are converted into consecutive integer indices using label encoders. Movie IDs are shifted by one so that index 0 can be reserved for padding in historical interaction sequences. User-side categorical attributes, including gender, age, and occupation, are also encoded into numerical indices. These encoded values are later used as inputs to embedding layers in the User Tower.

The system then processes movie genre information by constructing a genre lookup matrix. Each movie can belong to multiple genres, so every genre string is mapped to a genre ID. For each movie, its genre IDs are stored in a fixed-length vector with a maximum length of 5. If a movie has fewer genres, the remaining positions are padded with 0. This genre matrix allows the Item Tower to efficiently retrieve the genre features of each target movie during training and inference, as illustrated in Figure 7.

To build sequential recommendation samples, the system keeps only positive interactions, defined as ratings greater than or equal to 3. For each user, the most recent 20 positive movies are used as the history sequence. Let the positive interaction sequence of user u be $\mathcal{S}_u = (m_1, m_2, \dots, m_{T_u})$, where T_u is the number of positive interactions for user u .

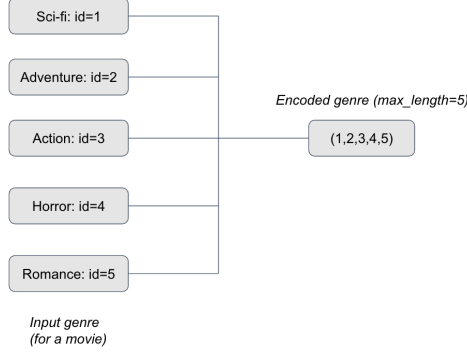


Figure 7: Preprocessing genres for movies in the dataset.

The fixed-length history sequence is constructed as

$$\mathcal{H}_u = \begin{cases} (m_{T_u-19}, \dots, m_{T_u}), & T_u \geq 20, \\ \underbrace{(0, \dots, 0)}_{20-T_u}, m_1, \dots, m_{T_u}, & T_u < 20, \end{cases} \quad (47)$$

where 0 denotes the padding index reserved during preprocessing.

For each movie m_t in the history sequence, the corresponding genre information is also stored. Let the genre list of movie m_t be

$$\mathcal{G}_{m_t} = (g_{t,1}, g_{t,2}, \dots, g_{t,L_g}), \quad (48)$$

where $g_{t,j}$ denotes the j -th genre ID associated with movie m_t and L_g is the actual number of genres for that movie. Each genre list is converted into a fixed-length vector of size 5:

$$\tilde{\mathcal{G}}_{m_t} = \begin{cases} (g_{t,1}, \dots, g_{t,5}), & L_g \geq 5, \\ (g_{t,1}, \dots, g_{t,L_g}, \underbrace{0, \dots, 0}_{5-L_g}), & L_g < 5. \end{cases} \quad (49)$$

Therefore, the genre representation for the 20-movie history is constructed in the same fixed-length manner as the movie history:

$$\mathcal{G}_u = \begin{cases} (\tilde{\mathcal{G}}_{m_{T_u-19}}, \dots, \tilde{\mathcal{G}}_{m_{T_u}}), & T_u \geq 20, \\ \underbrace{(0, \dots, 0)}_{20-T_u}, \tilde{\mathcal{G}}_{m_1}, \dots, \tilde{\mathcal{G}}_{m_{T_u}}, & T_u < 20, \end{cases} \quad (50)$$

which forms a fixed-size genre lookup structure aligned with the user's movie history. These 20-movie sequences provide the behavioral signal used by the User Tower to capture shifting tastes, dynamic preferences, and evolving viewing habits over time.

The final output of the preprocessing stage consists of processed user features, movie features, fixed-length history sequences, aligned genre sequences, and positive user–movie interaction samples. The train, validation, and test split strategies are described separately in Section 4.1, because they belong to the evaluation protocol rather than the model architecture itself.

3.6.4. Architecture

User Tower. The User Tower is responsible for encoding user-side information into a dense representation in the shared latent space. In this system, each user is represented by a combination of profile features and historical movie

interactions. The profile features include the encoded user ID, gender, age, and occupation, while the behavioral feature is a fixed-length sequence of previously interacted movies.

Following common neural collaborative filtering practice, each categorical input is first mapped into a trainable embedding vector [15]. The user ID is represented by a 64-dimensional embedding, while gender, age, and occupation are represented by lower-dimensional embeddings. Formally, the embedding matrices can be defined as

$$\mathbf{E}^{(u)} \in \mathbb{R}^{N_u \times 64}, \quad \mathbf{E}^{(g)} \in \mathbb{R}^{N_g \times 8}, \quad \mathbf{E}^{(a)} \in \mathbb{R}^{N_a \times 8}, \quad \mathbf{E}^{(o)} \in \mathbb{R}^{N_o \times 16}, \quad (51)$$

where N_u , N_g , N_a , and N_o denote the number of unique users, gender categories, age groups, and occupation categories after preprocessing. For MovieLens, gender, age, and occupation contain 2, 7, and 21 categories, respectively, while N_u depends on the number of users in the dataset. The corresponding embedding lookup for the user profile features can be written as

$$\mathbf{e}_u = \mathbf{E}^{(u)}[u], \quad \mathbf{e}_g = \mathbf{E}^{(g)}[g], \quad \mathbf{e}_a = \mathbf{E}^{(a)}[a], \quad \mathbf{e}_o = \mathbf{E}^{(o)}[o], \quad (52)$$

where u , g , a , and o denote the encoded user ID, gender, age, and occupation, respectively.

The fixed-length movie history sequence $\mathcal{H}_u = (\tilde{m}_1, \tilde{m}_2, \dots, \tilde{m}_{20})$ is embedded using the shared movie embedding matrix, following the standard item-sequence modeling setup used in sequential recommendation [16]. Let N_m denote the number of unique movies after preprocessing. Since movie ID 0 is reserved for padding, the movie embedding matrix contains one additional row and is defined as $\mathbf{E}^{(m)} \in \mathbb{R}^{(N_m+1) \times 64}$. Thus, each real movie and the padding token are mapped to a 64-dimensional vector. Since the same movie embedding table is also used by the Item Tower, the model learns a consistent representation for movies whether they appear as historical interactions or as target recommendation candidates.

Positional information is added through a learnable positional embedding matrix $\mathbf{P} \in \mathbb{R}^{20 \times 64}$, where the 20 rows correspond to the maximum history length. The embedded representation at position t is computed as

$$\mathbf{x}_t = \mathbf{E}^{(m)}[\tilde{m}_t] + \mathbf{P}[t], \quad t = 1, 2, \dots, 20, \quad (53)$$

where $\tilde{m}_t$ is the movie ID at position t . The full embedded history can therefore be written as

$$\mathbf{X}_u = [\mathbf{x}_1; \mathbf{x}_2; \dots; \mathbf{x}_{20}] \in \mathbb{R}^{20 \times 64}. \quad (54)$$

The embedded history sequence is then passed through a Transformer Encoder to model dependencies among the movies in the user's interaction history:

$$\mathbf{Z}_u = \text{TransformerEncoder}(\mathbf{X}_u), \quad \mathbf{Z}_u \in \mathbb{R}^{20 \times 64}. \quad (55)$$

The encoder-only Transformer uses multi-head self-attention to model dependencies among movies in the user's interaction history, enabling the User Tower to capture sequential preference patterns [17, 16, 18]. Unlike simple averaging, which treats historical movies as independent and equally informative, self-attention updates each movie representation by considering its relationships with the other movies in the sequence. Multi-head self-attention further improves this process by learning several attention patterns in parallel, where different heads can focus on different semantic aspects of the user's viewing history, such as genre consistency, recent interests, or long-term preference signals [17].

In this architecture, the encoder-only Transformer operates only on the encoded history sequence and produces a contextualized representation for each position. Each output vector therefore contains information from both the movie at that position and the surrounding historical interactions. A padding mask is applied so that padded positions do not contribute meaningful information to the attention computation or the final pooled representation. The contextualized sequence is then aggregated into a single history vector using masked mean pooling, which summarizes the user's historical behavior before being fed to the MLP layer.

As illustrated in Figure 8, the input to the MLP layer is constructed by concatenating all user-side feature representations into a single dense vector. Specifically, the 64-dimensional user ID embedding, the 8-dimensional gender

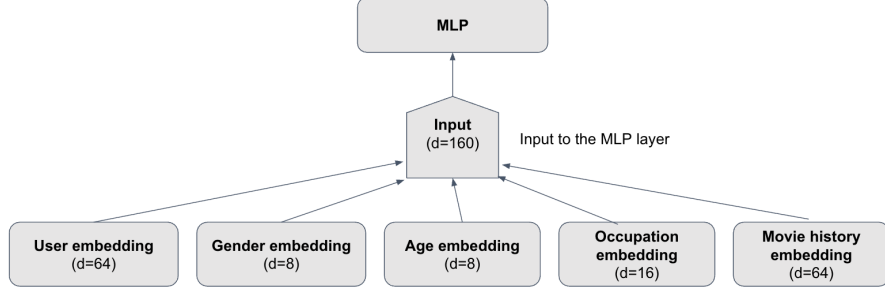


Figure 8: Input to the MLP layer after embedding the necessary user-side features.

embedding, the 8-dimensional age embedding, the 16-dimensional occupation embedding, and the 64-dimensional pooled history representation are combined as

$$\mathbf{c}_u = [\mathbf{e}_u; \mathbf{e}_g; \mathbf{e}_a; \mathbf{e}_o; \mathbf{h}_u] \in \mathbb{R}^{160}. \quad (56)$$

The MLP layer is customizable in depth. In this implementation, a two-layer MLP maps the concatenated user feature vector from 160 dimensions to 128 dimensions and then to 64 dimensions. Layer normalization, GELU activation, and dropout are used between the linear transformations to improve training stability and reduce overfitting [19, 20, 21]. The final output is L2-normalized to produce the user embedding $\mathbf{p}_u \in \mathbb{R}^{64}$.

Item Tower. The Item Tower is responsible for encoding each candidate movie into the same 64-dimensional latent space as the User Tower. In this implementation, an item is represented using two sources of information: its encoded movie ID and its associated genre labels. The overall structure is illustrated in Figure 9.

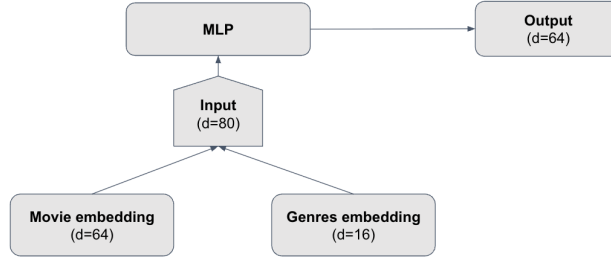


Figure 9: Overview of the Item Tower architecture.

Let i denote the encoded movie ID of a target candidate movie. The movie ID is first mapped through the shared movie embedding matrix used previously in the User Tower:

$$\mathbf{v}_i = \mathbf{E}^{(m)}[i], \quad \mathbf{v}_i \in \mathbb{R}^{64}. \quad (57)$$

For genre information, each movie is associated with a fixed-length list of genre IDs. Following the genre representation defined in Equations (48) and (49), the genre sequence for item i is represented as $\mathcal{G}_i = (g_{i,1}, g_{i,2}, \dots, g_{i,5})$. Each genre ID $g_{i,j}$ is mapped to a 16-dimensional embedding $\mathbf{e}_{i,j}^G = \mathbf{E}^G[g_{i,j}]$, where $\mathbf{E}^G \in \mathbb{R}^{|\mathcal{V}_G| \times 16}$ is the genre embedding matrix. The genre representation of movie i is then obtained by masked average pooling over the non-padding genre embeddings:

$$\mathbf{v}_i^G = \frac{1}{|\mathcal{G}_i^+|} \sum_{j: g_{i,j} \neq 0} \mathbf{e}_{i,j}^G, \quad \mathbf{v}_i^G \in \mathbb{R}^{16}, \quad (58)$$

where $\mathcal{G}_i^+$ denotes the set of valid genres.

Together, the movie ID embedding and the pooled genre representation form the item-side feature vector

$$\mathbf{c}_i = [\mathbf{v}_i; \mathbf{v}_i^G] \in \mathbb{R}^{80}. \quad (59)$$

This vector is passed through an MLP projection layer, which maps the 80-dimensional item feature representation to the shared 64-dimensional latent space used by both towers. Similar to the User Tower, layer normalization, GELU activation, and dropout are applied between linear layers to improve training stability and reduce overfitting. The final item embedding is then L2-normalized, producing $\mathbf{q}_i \in \mathbb{R}^{64}$, so that its dot product with the normalized user embedding can be interpreted as cosine similarity during retrieval.

3.6.5. Similarity Scoring and Training Objective

Given a mini-batch of B positive user–item pairs (u_b, i_b) , the User Tower produces a normalized user embedding $\mathbf{p}_b \in \mathbb{R}^{64}$ and the Item Tower produces a normalized item embedding $\mathbf{q}_b \in \mathbb{R}^{64}$. The similarity between each user embedding and each item embedding in the batch is calculated as

$$S_{b,k} = \frac{\mathbf{p}_b^\top \mathbf{q}_k}{\tau}, \quad (60)$$

where τ is the temperature parameter. Since both embeddings are L2-normalized, the dot product is equivalent to cosine similarity.

The model is optimized using the in-batch softmax loss. For each user u_b , the matched item i_b is treated as the positive target, while the other items in the same mini-batch are used as implicit negative samples. The loss for each batch is defined as

$$\mathcal{L} = -\frac{1}{B} \sum_{b=1}^B \log \frac{\exp(S_{b,b})}{\sum_{k=1}^B \exp(S_{b,k})}. \quad (61)$$

This objective encourages matched user–item pairs to have higher similarity than mismatched pairs in the same batch.

3.6.6. Inference Procedure

During inference, the Item Tower is applied to all candidate movies to obtain a matrix of item embeddings. These embeddings can be precomputed because they do not depend on the target user. For a given user, the User Tower produces a single user vector, and recommendation scores are obtained by computing dot products between this vector and all candidate item vectors. Movies that already appear in the user’s interaction history are masked out, and the remaining movies are sorted to produce the final Top- K recommendation list.

3.7. LightGCN

3.7.1. Overview

Collaborative Filtering (CF) is a fundamental technique for personalized recommender systems, relying on historical user-item interactions to predict future preferences. Recently, Graph Convolutional Networks (GCNs) have shown significant promise in CF by modeling the user-item interaction matrix as a bipartite graph [9]. However, standard GCN architectures inherit heavy components such as non-linear activation functions and feature transformation weight matrices from node classification tasks, which are proven to be overly complex and even detrimental for CF tasks where node attributes are often just unique identifiers [7].

As illustrated in the overview from figure 10, the model architecture replaces deep parameterized layers with a pair of highly efficient, topology-driven mechanisms. In the context of the MovieLens-1M user-movie interaction graph:

- **Light Graph Convolution (LGC):** Performs a normalized sum over direct neighbors across successive message-passing layers (such as Layer 1 through Layer 3). This step iteratively refines raw User and Movie ID embeddings by smoothing them with the representations of the movies a user has watched, and the users a movie has been watched by.
- **Layer Combination:** Aggregates the distinct representations generated at each convolution step via a weighted sum ($\oplus$). This ensures that the final embedding captures both localized, direct preferences (lower-order layers) and broad, transitive collaborative signals (higher-order layers, such as "users who liked what you liked").

The defining strategic advantage of this architecture lies in its linear simplicity and explicit structural inductive bias. By stripping graph convolution down to its absolute essence—eliminating heavy neural transformations—LightGCN bypasses the risks of over-parameterization and overfitting on sparse interaction data. Prediction reduces to a highly optimized dot product ($\otimes$) between the final, smoothed user and item vectors. Within this comparative framework, LightGCN represents the graph-based paradigm, demonstrating how the innate geometric topology of historical interactions can be engineered to capture complex user preferences natively without a heavy neural stack.

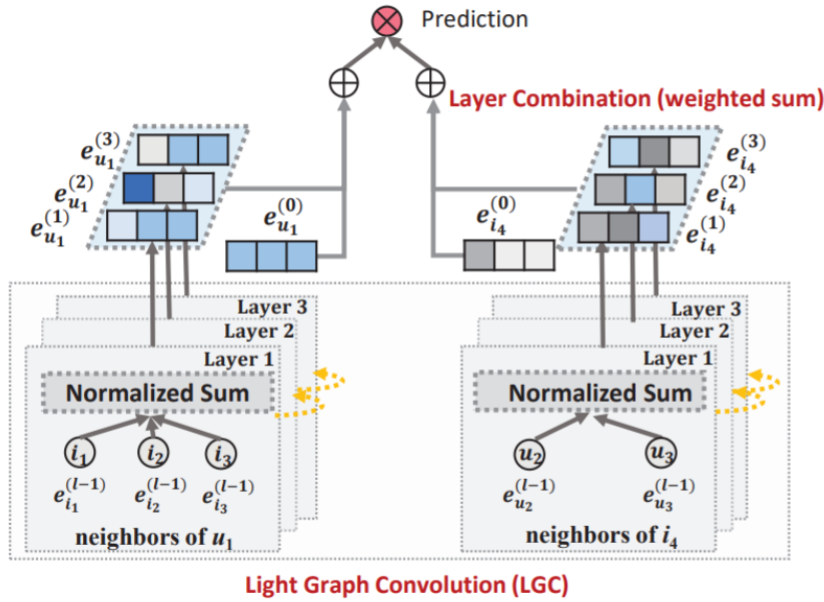


Figure 10: Overview of the LightGCN architecture and message-passing pipeline.

3.7.2. Pipeline

The proposed system follows a structured retrieval and training pipeline designed for scalable collaborative filtering:

- **Data Preprocessing:** Raw MovieLens data is parsed to construct a bipartite interaction graph. This involves extracting positive implicit feedback, ensuring density via k -core filtering, and mapping users and items to contiguous latent spaces.
- **Graph Construction:** The bipartite graph is pre-computed and converted into an undirected format with self-loops removed. Symmetric normalization is implicitly applied via PyTorch Geometric’s LGConv module [10].
- **Representation Learning:** The LightGCN model linearly propagates initial embeddings (layer-0) across the graph topology for K layers. The embeddings from all layers are then stacked and mean-pooled to form the final multi-scale representations for both users and items.

- **Optimization:** A continuous negative sampling strategy creates training triplets of (u, i, j) (user, positive item, negative item). The model is optimized using the Bayesian Personalized Ranking (BPR) loss [11], accompanied by L_2 regularization specifically applied to the layer-0 embeddings to prevent overfitting.
- **Inference:** Recommendation lists are generated using a highly vectorized dot-product scoring mechanism between the target user and the entire item catalog, masking historical interactions before top- K extraction.

3.7.3. Data Preprocessing

Let $\mathcal{U}$ be the set of users and $\mathcal{I}$ be the set of items. The raw dataset contains explicit ratings on a scale of 1 to 5. To formulate the implicit feedback setup commonly used in CF, we filter the dataset to retain only positive interactions. Specifically, any rating $r_{ui} \geq 3.0$ is mapped to $y_{ui} = 1$, and all other interactions are treated as unobserved.

To ensure data integrity and sufficient connectivity for message passing, a strict k -core filter is applied with $k = 5$. The system iteratively removes users who have fewer than 5 positive interactions. This filtering is crucial for Graph Convolutional Networks, as nodes with too few connections (cold-start nodes) lack sufficient topological structure to benefit from message passing and can add noise to the embeddings of their neighbors. Following this filtering, the raw user ID and movie ID strings are mapped into zero-indexed integer sequences using label factorization. This mapping allows the IDs to directly serve as indices for the embedding lookup tables.

The user-item interaction matrix $R \in \mathbb{R}^{|\mathcal{U}| \times |\mathcal{I}|}$ is defined as $R_{ui} = 1$ if $y_{ui} = 1$, and 0 otherwise. The data is partitioned into training and testing sets either using a stratified random split or a temporal leave-one-out approach based on timestamps. The training interactions are then extracted into an edge index tensor $\mathcal{E}_{train} \in \mathbb{Z}^{2 \times |\mathcal{E}|}$, which is utilized to build the graph topology.

During the training phase, dynamic negative sampling is performed by a custom BPRDataset. For each observed interaction $(u, i) \in \mathcal{E}_{train}$, the dataset samples a negative item $j \in \mathcal{I}$ uniformly at random such that $(u, j) \notin \mathcal{E}_{train}$.

3.7.4. Architecture

The model architecture, directly implements the LightGCN paradigm [7].

Embedding Layer. Users and items are assigned dense embeddings in a shared latent space of dimension d (we use $d = 64$ in our implementation). Let $\mathbf{E}^{(0)} \in \mathbb{R}^{(|\mathcal{U}|+|\mathcal{I}|) \times d}$ denote the base embedding matrix. These embeddings are initialized using a normal distribution $\mathcal{N}(0, 0.1^2)$. For a given user u and item i , their initial layer-0 embeddings are denoted as $\mathbf{e}_u^{(0)}$ and $\mathbf{e}_i^{(0)}$.

Light Graph Convolution (LGConv). Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the undirected bipartite graph where $\mathcal{V} = \mathcal{U} \cup \mathcal{I}$. The adjacency matrix of this graph can be formulated as:

$$A = \begin{pmatrix} 0 & R \\ R^\top & 0 \end{pmatrix} \quad (62)$$

Unlike standard Graph Convolutional Networks [12] which use $H^{(k+1)} = \sigma(\tilde{A}H^{(k)}W^{(k)})$, LightGCN completely drops the non-linear activation function $\sigma(\cdot)$ and the feature transformation matrix $W^{(k)}$. At the k -th layer, the embedding of user u is updated by aggregating the representations of its connected neighbors (items):

$$\mathbf{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|} \sqrt{|\mathcal{N}_i|}} \mathbf{e}_i^{(k)} \quad (63)$$

where $\mathcal{N}_u$ and $\mathcal{N}_i$ denote the sets of neighbors for user u and item i , respectively. In matrix form, the propagation rule is:

$$\mathbf{E}^{(k+1)} = \left(D^{-\frac{1}{2}}AD^{-\frac{1}{2}}\right)\mathbf{E}^{(k)} \quad (64)$$

where D is the diagonal degree matrix with $D_{ii} = |\mathcal{N}_i|$. This symmetric normalization prevents the scale of embeddings from exploding with graph convolutions. This operation is efficiently computed utilizing PyTorch Geometric’s LGConv.

Layer Aggregation. After K layers of graph convolution (we configure $K = 2$ for temporal splits and $K = 3$ for random splits), the model gathers the embeddings from all layers $\{\mathbf{E}^{(0)}, \mathbf{E}^{(1)}, \dots, \mathbf{E}^{(K)}\}$. To capture the structural semantics at different receptive fields, LightGCN combines them to form the final node representations. In this implementation, the combination is performed using a mean-pooling mechanism:

$$\mathbf{e}_u = \frac{1}{K+1} \sum_{k=0}^K \mathbf{e}_u^{(k)}, \quad \mathbf{e}_i = \frac{1}{K+1} \sum_{k=0}^K \mathbf{e}_i^{(k)} \quad (65)$$

3.7.5. Training Objective

The model is optimized using the Bayesian Personalized Ranking (BPR) loss [11], a widely adopted pairwise loss function for implicit feedback. BPR assumes that a user prefers an observed (interacted) item over an unobserved item.

For a batch of sampled triplets (u, i, j) where $i \in \mathcal{N}_u$ and $j \notin \mathcal{N}_u$, the model computes the affinity scores via inner product:

$$\hat{y}_{ui} = \mathbf{e}_u^\top \mathbf{e}_i, \quad \hat{y}_{uj} = \mathbf{e}_u^\top \mathbf{e}_j \quad (66)$$

The BPR loss is then formulated to maximize the margin between the positive and negative scores:

$$\mathcal{L}_{BPR} = -\frac{1}{|\mathcal{B}|} \sum_{(u,i,j) \in \mathcal{B}} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) \quad (67)$$

where $\sigma(\cdot)$ is the sigmoid function and $\mathcal{B}$ represents the mini-batch.

To prevent overfitting, L_2 regularization is applied. Crucially, as advocated by [7], the regularization is applied *only* to the initial layer-0 embeddings, rather than the final representations, because the higher-layer embeddings are deterministic linear combinations of layer-0. The regularization term is:

$$\mathcal{L}_{reg} = \frac{1}{2|\mathcal{B}|} \sum_{(u,i,j) \in \mathcal{B}} (\|\mathbf{e}_u^{(0)}\|^2 + \|\mathbf{e}_i^{(0)}\|^2 + \|\mathbf{e}_j^{(0)}\|^2) \quad (68)$$

The final objective function minimized by the Adam optimizer is the sum of the BPR loss and the weighted regularization term:

$$\mathcal{L} = \mathcal{L}_{BPR} + \lambda \mathcal{L}_{reg} \quad (69)$$

where we set the regularization weight to $\lambda = 10^{-4}$. Due to its simplified structure without feature transformations, the time complexity of the layer propagation is $O(|\mathcal{E}|d)$, which scales linearly with the graph size and ensures efficient training.

3.7.6. Inference

During the inference phase, the objective is to generate a ranked list of Top- K items for a target user u . The inference procedure is fully vectorized to efficiently handle the large candidate space.

1. **Embedding Retrieval:** The learned final user embedding vector $\mathbf{e}_u \in \mathbb{R}^d$ and the full item embedding matrix $\mathbf{E}_I \in \mathbb{R}^{|\mathcal{I}| \times d}$ are extracted from the optimal model state.

2. **Affinity Scoring:** The recommendation scores over all items are calculated simultaneously via matrix-vector multiplication:

$$\hat{\mathbf{y}}_u = \mathbf{E}_I \mathbf{e}_u \quad (70)$$

where $\hat{\mathbf{y}}_u \in \mathbb{R}^{|I|}$.

3. **Interaction Masking:** To ensure the model recommends novel items, the affinity scores for items the user has already interacted with during training are masked by setting their values to $-\infty$.
4. **Top-K Retrieval:** The indices of the highest K scores in the masked vector $\hat{\mathbf{y}}_u$ are extracted.
5. **Metadata Integration:** The predicted item indices are inverse-transformed back to the original movie IDs. These are joined with the movies metadata file to provide human-readable recommendations, including the movie titles and genres.

3.8. Self-Attentive Sequential Recommendation (SASRec)

3.8.1. Overview

SASRec is a sequential recommendation model that uses self-attention to predict a user’s next likely interaction from their ordered behavior history [16]. Unlike matrix factorization and graph-based collaborative filtering, which primarily model users through unordered interaction sets, SASRec treats the user history as a time-ordered sequence. This makes it especially suitable for settings where recent interactions reveal short-term intent, while earlier interactions still provide useful long-term preference context.

In the MovieLens-1M setting, each user is represented by a chronological sequence of positively rated movies. The model learns to encode this sequence using Transformer-style masked self-attention [17]. At each position, SASRec attends only to previous positions, preventing information leakage from future interactions and aligning the training objective with next-item prediction.

The key components of the approach are:

- **Sequential User Representation:** Each user’s watched-movie history is sorted by timestamp and converted into fixed-length item sequences using truncation and padding.
- **Causal Self-Attention:** A stack of self-attention blocks contextualizes each movie representation using only earlier interactions in the same user sequence.
- **Next-Item Prediction:** The contextualized representation at each time step is trained to score the next observed movie above sampled negative movies.

The defining advantage of SASRec is its ability to adapt the recommendation signal according to the user’s current trajectory. Instead of compressing all historical interactions into a single static user vector, the model produces position-dependent representations that can emphasize recent behavior when short-term intent is informative and preserve older signals when they remain relevant.

3.8.2. Pipeline

The proposed SASRec system follows a sequential training and inference pipeline:

- **Data Preprocessing:** Raw MovieLens ratings are converted into implicit positive interactions, sorted by timestamp for each user, and mapped to contiguous user, movie, and genre indices. Movie and genre index 0 are reserved for padding.

- **Sequence Construction:** For each user, sliding windows are generated from the chronological interaction sequence. The input sequence contains previous movies and their aligned genre features, while the target sequence contains the next movie at each position.
- **Representation Learning:** Item embeddings, genre embeddings, and positional embeddings are summed and passed through causal self-attention blocks to produce contextualized sequence representations.
- **Optimization:** The model is trained with full-catalog softmax cross-entropy using `nn.CrossEntropyLoss`. At each valid sequence position, the hidden state is scored against all N_i movies, avoiding negative sampling during optimization.
- **Inference:** For each user, the most recent interaction sequence and corresponding genre sequence are encoded, all candidate movies are scored against the final hidden state, and previously seen movies are masked before top- K ranking.

3.8.3. Data Preprocessing

Let $\mathcal{U}$ denote the set of users and $\mathcal{I}$ denote the set of movies. The explicit MovieLens-1M ratings are first converted into implicit feedback by retaining positive interactions. In this implementation, ratings greater than or equal to 3 are treated as positive signals, while lower ratings are excluded from the sequential history.

For each user u , the positive interactions are sorted by timestamp to form an ordered sequence

$$\mathcal{S}_u = (i_1, i_2, \dots, i_{T_u}), \quad (71)$$

where $i_t \in \mathcal{I}$ is the movie interacted with at time step t and T_u is the number of positive interactions for user u . Raw movie IDs are mapped to contiguous integer indices, with index 0 reserved for padding.

Given a maximum sequence length L , the model constructs fixed-length input sequences by keeping the most recent L interactions before the prediction point. For a prediction target at time $t + 1$, the input sequence is

$$\mathbf{s}_{u,t} = (i_a, i_{a+1}, \dots, i_t), \quad a = \max(1, t - L + 1), \quad (72)$$

left-padded with zeros when fewer than L previous interactions are available. The corresponding positive target is i_{t+1} . This sequence construction preserves temporal order and ensures that the model never observes future interactions when predicting the next movie.

For temporal evaluation, the final interaction of each user is used as the test target, the second-to-last interaction is used for validation, and earlier interactions are used for training. This split matches the next-item recommendation objective and evaluates whether the model can extrapolate from a user's past behavior to their future behavior.

3.8.4. Architecture

SASRec is an encoder-style sequential model built from item embeddings, genre embeddings, positional embeddings, and stacked causal self-attention blocks. Let N_i be the number of movies after preprocessing, let N_g be the number of genre categories, and let d be the hidden dimension. The item embedding matrix is

$$\mathbf{E} \in \mathbb{R}^{(N_i+1) \times d}, \quad (73)$$

where the extra row corresponds to the padding index. To inject item-side semantic information into the sequential representation, the model also maintains a genre embedding matrix

$$\mathbf{G} \in \mathbb{R}^{(N_g+1) \times d}, \quad (74)$$

where the additional row is reserved for padded or missing genre entries. A learnable positional embedding matrix

$$\mathbf{P} \in \mathbb{R}^{L \times d} \quad (75)$$

encodes the position of each item in the fixed-length sequence.

Embedding Layer. For an input sequence $\mathbf{s}_{u,t} = (s_1, s_2, \dots, s_L)$, let g_ℓ denote the encoded genre feature associated with the movie at position ℓ . The initial representation at position ℓ is computed as the sum of the item embedding, the genre embedding, and the positional embedding:

$$\mathbf{x}_\ell = \mathbf{E}[s_\ell] + \mathbf{G}[g_\ell] + \mathbf{P}[\ell], \quad \ell = 1, 2, \dots, L. \quad (76)$$

This formulation preserves the sequential inductive bias of SASRec while enriching each interaction with explicit movie-genre information from the MovieLens metadata. Padding positions are masked so that they do not contribute to attention or loss computation.

Causal Self-Attention Blocks. The embedded sequence is passed through stacked Transformer blocks. In each block, query, key, and value projections are computed from the previous layer representation. Scaled dot-product attention is applied as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{M}\right)\mathbf{V}, \quad (77)$$

where $\mathbf{M}$ is a causal mask that assigns invalid attention scores to future positions and padding positions. This mask enforces the autoregressive structure: the representation at position ℓ can use information from positions 1 through ℓ , but not from positions after ℓ .

Feed-Forward Layer. After self-attention, each position is processed by a point-wise feed-forward network with residual connections and normalization. This component refines the contextualized item representation independently at each position while preserving the temporal structure introduced by the attention mask.

Let $\mathbf{h}_\ell \in \mathbb{R}^d$ denote the final hidden representation at position ℓ . This vector summarizes the user’s observed sequence up to that position and is used to predict the next movie.

3.8.5. Training Objective

SASRec is trained as a next-item prediction model using full-catalog softmax cross-entropy. For each valid position ℓ , the positive item is the next observed movie $i_{\ell+1}$. Instead of drawing sampled negative items, the implementation computes the target score against every movie in the catalog. This choice is practical for MovieLens-1M because the item catalog is sufficiently dense and moderate in size, allowing the model to optimize over all N_i candidate movies directly.

The score vector for position ℓ is computed by comparing the hidden state $\mathbf{h}_\ell$ with the complete item embedding table:

$$\hat{\mathbf{y}}_\ell = \mathbf{h}_\ell^\top \mathbf{E}_{1:N_i}^\top \in \mathbb{R}^{N_i}, \quad (78)$$

where $\mathbf{E}_{1:N_i}$ excludes the padding row and contains the embeddings of all real movies. The probability assigned to candidate movie j is therefore

$$p_{\ell j} = \frac{\exp(\hat{y}_{\ell j})}{\sum_{k=1}^{N_i} \exp(\hat{y}_{\ell k})}. \quad (79)$$

Using full-catalog cross-entropy, the loss for user u can be written as

$$\mathcal{L}_u = - \sum_{\ell \in \mathcal{V}_u} \log \frac{\exp(\hat{y}_{\ell i_{\ell+1}})}{\sum_{k=1}^{N_i} \exp(\hat{y}_{\ell k})}, \quad (80)$$

where $\mathcal{V}_u$ is the set of non-padding positions for user u . This objective corresponds to the standard `nn.CrossEntropyLoss` formulation: the logits are the full catalog scores $\hat{\mathbf{y}}_\ell$, and the class label is the index of the true next movie $i_{\ell+1}$. By normalizing over the entire movie catalog, the model avoids negative sampling during optimization and directly learns to rank the observed next item against all available alternatives. The total objective sums this loss over all users and adds regularization to reduce overfitting.

3.8.6. Inference

During inference, the model constructs a fixed-length sequence from the user’s most recent positive interactions and feeds it through the trained SASRec encoder. The final non-padding hidden state is treated as the current user-intent representation:

$$\mathbf{z}_u = \mathbf{h}_L^{\text{final}}, \quad (81)$$

assuming the input has been left-padded so that the latest observed interaction appears at the final position.

All candidate movies are scored by dot product with $\mathbf{z}_u$:

$$\hat{y}_{uj} = \mathbf{z}_u^\top \mathbf{E}[j], \quad j \in \mathcal{I}. \quad (82)$$

Movies already observed in the user’s history are masked to prevent recommending previously consumed items. The remaining candidates are sorted by score, and the top- K movies are returned as the recommendation list. This inference procedure directly reflects the sequential objective: the model recommends items that are most compatible with the user’s latest contextualized behavior state.

4. Experiments

4.1. Setup

To comprehensively evaluate the recommendation models formulated in Section 3, we employ two distinct data splitting strategies, each mirroring a different real-world scenario. First, we use a **Random 80-20 Split**, which randomly partitions the user-item interactions into training and testing sets. This evaluates the models’ ability to capture general, global collaborative signals. Second, we use a **Temporal Leave-one-out Split**, where the most recent interaction for each user is held out for testing, and all preceding interactions form the training set. This evaluates the models’ ability to predict the immediate next item based on sequential dynamics.

To assess the quality of the generated recommendation lists, we rely on standard information retrieval metrics tailored for recommender systems [1]. Specifically:

- **Precision@K**: Measures the proportion of items in the top- K recommendations that the user actually interacted with.
- **Recall@K**: Measures the proportion of all relevant (interacted) items that successfully appeared in the top- K list.
- **HitRate@K**: A boolean metric evaluating whether the single held-out test item was successfully retrieved within the top- K recommendations.
- **Mean Reciprocal Rank (MRR@K)**: Evaluates the ranking quality by taking the inverse of the rank of the first relevant item. It strongly penalizes models that place relevant items lower in the list.
- **Normalized Discounted Cumulative Gain (NDCG@K)**: A position-aware ranking metric that assigns higher scores to hits at the very top of the ranking compared to hits further down, normalized against the ideal perfect ranking.

For the random split, we evaluate Top-20 recommendations ($K = 20$) using NDCG@20, Recall@20, MRR@20, and Precision@20. For the leave-one-out split, we evaluate Top-10 recommendations ($K = 10$) using NDCG@10, HitRate@10, and MRR@10.

4.2. Results

The aggregate performance of all benchmarked architectures (detailed in Section 3) under the Random Split is presented in Table 1. The results for the Temporal Leave-one-out Split are detailed in Table 2.

These empirical results directly reflect the architectural inductive biases of the models discussed in the methodology. Specifically, we observe how variations in interaction modeling - from simple dot products in Matrix Factorization to graph-based message passing in LightGCN - affect ranking performance under different validation constraints.

Model	NDCG@20	Recall@20	MRR@20	Precision@20
Content-based Filtering	0.0185 ± 0.0006	0.0115 ± 0.0006	0.0541 ± 0.0016	0.0152 ± 0.0004
Hybrid Filtering	0.0474 ± 0.0047	0.0518 ± 0.0095	0.1004 ± 0.0071	0.0302 ± 0.0048
Memory-based Filtering	0.0363 ± 0.0008	0.0148 ± 0.0007	0.1014 ± 0.0018	0.0274 ± 0.0012
Matrix Factorization	0.1469 ± 0.0010	0.1275 ± 0.0015	0.2889 ± 0.0038	0.1055 ± 0.0010
Neural CF	0.3264 ± 0.0006	0.2268 ± 0.0006	0.5734 ± 0.0006	0.2460 ± 0.0006
Two Towers	0.0180 ± 0.0003	0.0495 ± 0.0008	0.0096 ± 0.0002	0.0025 ± 0.0001
LightGCN	0.3578 ± 0.0011	0.2604 ± 0.0012	0.6077 ± 0.0022	0.2630 ± 0.0009

Table 1: Model Performance under Random 80-20 Split ($K = 20$)

Model	NDCG@10	HitRate@10	MRR@10
Content-based Filtering	0.0054 ± 0.0003	0.0108 ± 0.0005	0.0037 ± 0.0007
Hybrid Filtering	0.0086 ± 0.0005	0.0192 ± 0.0005	0.0054 ± 0.0003
Memory-based Filtering	0.0020 ± 0.0002	0.0051 ± 0.0002	0.0011 ± 0.0001
Matrix Factorization	0.0109 ± 0.0004	0.0228 ± 0.0021	0.0073 ± 0.0011
Neural CF	0.0148 ± 0.0004	0.0329 ± 0.0010	0.0094 ± 0.0003
Two Towers	0.0980 ± 0.0024	0.1888 ± 0.0026	0.0706 ± 0.0025
LightGCN	0.0438 ± 0.0003	0.0899 ± 0.0013	0.0302 ± 0.0004
SASRec	0.1762 ± 0.0031	0.3074 ± 0.0045	0.1360 ± 0.0030

Table 2: Model Performance under Temporal Leave-one-out Split ($K = 10$)

5. Discussion

The experimental results highlight the profound impact that the chosen evaluation protocol (Random vs. Temporal) has on model performance, revealing the specific strengths, weaknesses, and practical trade-offs of different architectural paradigms.

5.1. Performance Disparities: Global vs. Sequential Contexts

Under the random 80-20 split (Table 1), the task fundamentally becomes one of pattern matching across the entire user-item interaction graph, ignoring chronological order. In this setting, **LightGCN** emerges as the dominant architecture across all metrics, achieving the highest NDCG@20 (0.3578), Recall@20 (0.2604), MRR@20 (0.6077), and Precision@20 (0.2630). By explicitly modeling high-order connectivity in the bipartite graph, LightGCN accurately captures deep, global collaborative signals that other models miss.

However, the performance hierarchy shifts dramatically under the temporal leave-one-out protocol (Table 2). When the goal is strictly predicting the *next* immediate item, static global models like LightGCN and Matrix Factorization

experience a severe drop in performance. LightGCN performs poorly here because graph aggregation washes out temporal dynamics - it weighs an item a user interacted with five years ago equally to one they interacted with yesterday.

Conversely, **SASRec** significantly outperforms all competitors in the temporal split, achieving an impressive NDCG@10 of 0.1762 and a HitRate@10 of 0.3074. SASRec’s self-attention mechanism is explicitly designed to weigh the importance of recent interactions against long-term preferences, allowing it to capture shifting user intents which graph models inherently ignore. Interestingly, the **Two-Tower** architecture, while performing abysmally on the random split (0.0180 NDCG@20), demonstrates surprising resilience in the temporal split (0.0980 NDCG@10), outperforming both Matrix Factorization and LightGCN. This suggests that the Two-Tower’s capacity to retrieve candidates based on localized sequence embeddings makes it relatively well-suited for strict time-based evaluation, even if its global collaborative power is weak compared to graph methods.

5.2. Evaluating the Baselines: When are they enough?

Classic **Matrix Factorization (MF)** serves as a highly robust baseline. While it is outperformed by deep models like Neural CF and LightGCN on random splits, it offers an excellent trade-off between performance and computational efficiency. MF is extremely fast to train, highly scalable, and easy to serve in production. For applications with strict latency or compute constraints, MF is often “good enough.”

On the other hand, purely **Memory-based** and **Content-based** approaches struggle significantly across both splits. Content-based models rely entirely on human-engineered metadata (e.g., genres), failing to discover latent relationships. Memory-based CF suffers heavily from data sparsity, as it relies on exact co-occurrences. The **Hybrid Filtering** approach successfully merges these two paradigms, consistently outperforming its individual content-based and memory-based sub-components across metrics. However, its performance still lags far behind deep collaborative methods. These simple baselines are not recommended for mature production systems unless used strictly to solve the cold-start problem where interaction data is completely absent.

5.3. Trade-offs and Training Difficulties

Achieving state-of-the-art performance with models like **Neural CF**, **LightGCN**, and **SASRec** comes at the cost of training complexity and resource consumption.

While Neural CF demonstrates strong results in the random split (achieving the second-highest NDCG@20 and Precision@20), it is notoriously difficult to train. The architecture is highly non-convex and sensitive to initialization. Achieving peak performance often requires independently pre-training the generalized matrix factorization (GMF) and multi-layer perceptron (MLP) components before fine-tuning them together. Consequently, it is often not strongly suggested for deployment if a simpler, more elegant graph model like LightGCN is available, as the training instability frequently outweighs the benefits over standard MF.

Similarly, SASRec requires complex sequential data pipelines and $O(N^2)$ memory complexity regarding sequence length due to the self-attention mechanism. LightGCN requires constructing and maintaining large, sparse adjacency matrices in memory and performing expensive message-passing steps.

5.4. The Impact of Dataset Characteristics

The inherent nature of the MovieLens-1M dataset plays a crucial role in dictating which baselines succeed and which fail. MovieLens-1M is exceptionally dense (roughly 4.19% density) compared to real-world industrial datasets which often operate at $> 99.9\%$ sparsity.

Because the interaction graph is so dense and highly connected, collaborative signals are extremely rich. This directly explains why graph-based models (LightGCN) and latent factor models (Matrix Factorization) perform so exceptionally well - there is an abundance of intersecting user-item paths to learn from.

Conversely, this density also explains why **Content-based** methods appear so weak in comparison. The available metadata (like movie genres) is extremely coarse and limited compared to the vast wealth of behavioral signals contained in the dense interaction matrix. If we were operating in a highly sparse environment (or a strict cold-start scenario), the performance gap would shrink significantly, as content-based methods would be the only reliable way to connect unobserved items. The dataset’s density naturally biases the results to heavily favor pure collaborative algorithms.

5.5. Conclusion

Ultimately, the choice of a recommendation system architecture must be heavily guided by the specific nature of the application and available infrastructure. For platforms where recommendations are driven by overall, long-term user tastes (e.g., suggesting new genres to explore), graph-based approaches like LightGCN offer state-of-the-art ranking metrics including Precision and Recall. For highly dynamic environments where user intent shifts rapidly from session to session, sequential models like SASRec are indispensable. However, practitioners must always weigh these performance gains against the increased training complexities, recognizing that robust baselines like Matrix Factorization often provide the most practical starting point.

References

- [1] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, *Recommender Systems Handbook*, Springer, 2011.
- [2] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” *Proceedings of the 10th International Conference on World Wide Web*, pp. 285–295, 2001.
- [3] P. Lops, M. de Gemmis, and G. Semeraro, “Content-based recommender systems: State of the art and trends,” in *Recommender Systems Handbook*, New York, NY: Springer, 2011, pp. 73–105.
- [4] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge, UK: Cambridge University Press, 2008.
- [5] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [6] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative filtering for implicit feedback datasets,” *Proceedings of the 8th IEEE International Conference on Data Mining*, pp. 263–272, 2008.
- [7] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, “LightGCN: Simplifying and powering graph convolution network for recommendation,” *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 639–648, 2020.
- [8] Harper, F. Maxwell, and Joseph A. Konstan. “The MovieLens Datasets: History and Context.” *ACM Transactions on Interactive Intelligent Systems (TiiS)* 5, no. 4 (2015): 1-19.
- [9] Wang, Xiang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. “Neural Graph Collaborative Filtering.” In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 165–174, 2019.
- [10] Fey, Matthias, and Jan Eric Lenssen. “Fast Graph Representation Learning with PyTorch Geometric.” In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [11] Rendle, Steffen, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. “BPR: Bayesian Personalized Ranking from Implicit Feedback.” In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 452–461, 2009.
- [12] Kipf, Thomas N., and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks.” In *International Conference on Learning Representations (ICLR)*, 2017.
- [13] P. Covington, J. Adams, and E. Sargin, “Deep neural networks for YouTube recommendations,” *Proceedings of the 10th ACM Conference on Recommender Systems*, pp. 191–198, 2016.
- [14] P.-S. Huang, X. He, J. Gao, L. Deng, A. Acero, and L. Heck, “Learning deep structured semantic models for web search using clickthrough data,” *Proceedings of the 22nd ACM International Conference on Information & Knowledge Management*, pp. 2333–2338, 2013.
- [15] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, “Neural collaborative filtering,” *Proceedings of the 26th International Conference on World Wide Web*, pp. 173–182, 2017.
- [16] W.-C. Kang and J. McAuley, “Self-attentive sequential recommendation,” *Proceedings of the 2018 IEEE International Conference on Data Mining*, pp. 197–206, 2018.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [18] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, “BERT4Rec: Sequential recommendation with bidirectional encoder representations from Transformer,” *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pp. 1441–1450, 2019.
- [19] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [20] D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” *arXiv preprint arXiv:1606.08415*, 2016.
- [21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

- [22] M. D. Ekstrand, J. T. Riedl, and J. A. Konstan, "Collaborative Filtering Recommender Systems," *Foundations and Trends® in Human-Computer Interaction*, vol. 4, no. 2, pp. 81–173, 2011.
- [23] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T. S. Chua, "Neural Collaborative Filtering," in *Proceedings of the 26th International Conference on World Wide Web*, 2017, pp. 173–182.